

AWS en pratique

Bases de données : RDS, Aurora, DynamoDB, secrets

RDS, créer et se connecter. RDS, disponibilité et sauvegardes. DynamoDB. Choisir sa base et protéger ses secrets.

RDS

moteurs, création

Disponibilité

Multi-AZ, réplicas

Sauvegardes

snapshots, restauration

DynamoDB

clés, capacité, index

Choisir

SQL, NoSQL, cache

Secrets

Secrets Manager, KMS

Support de formation, version 1.2, octobre 2026. Parcours « AWS en pratique ». Fait suite au support « Stockage et diffusion ».

RDS, créer et se connecter

- Citer les moteurs disponibles, et ce qu'AWS gère à votre place.
- Choisir classe d'instance, stockage, groupe de sous-réseaux et security group.
- Confier le mot de passe principal à Secrets Manager.

RDS, disponibilité et sauvegardes

- Distinguer Multi-AZ et réplicas de lecture.
- Utiliser sauvegardes automatiques, snapshots et restauration à un instant donné.
- Situer Aurora par rapport à RDS.

DynamoDB

- Décrire table, item, clé de partition et clé de tri.
- Choisir entre capacité à la demande et provisionnée.
- Préférer Query à Scan ; utiliser un index secondaire et le TTL.

Choisir sa base, protéger ses secrets

- Choisir entre relationnel et NoSQL d'après des critères.
- Dire à quoi sert ElastiCache.
- Répartir les rôles entre Secrets Manager, Parameter Store et KMS.

Prérequis. Les supports précédents : rôles IAM, VPC à trois niveaux, security groups chaînés, instances sans état, snapshots, AWS Backup. Des notions de SQL aident, sans être indispensables.

Fil rouge. Les données structurées de UtopiosShop : commandes et clients dans une base relationnelle, paniers et sessions dans DynamoDB, et aucun mot de passe dans le code.

RDS, créer et se connecter	
Amazon RDS	Moteurs, partage des tâches.
Créer une base	Classe, stockage, réseau, connexion.
Mot de passe géré	Secrets Manager, rotation.

DynamoDB	
DynamoDB	Table, item, clés.
Capacité	À la demande, provisionnée.
Requêtes et index	Query, Scan, index, TTL.

Disponibilité et sauvegardes	
Disponibilité	Multi-AZ, réplicas de lecture.
Sauvegardes	Automatiques, snapshots, instant donné.
Aurora	Stockage partagé, Serverless.

Choisir et protéger	
Choisir sa base	Critères, ElastiCache.
Secrets et clés	Secrets Manager, Parameter Store, KMS.

Convention. Les tarifs cités sont ceux de la région us-east-1, hors taxes, à titre d'ordre de grandeur. La page de tarification de chaque service fait foi.

RDS, créer et se connecter, partie 1

Amazon RDS

Une base de données relationnelle dont AWS assure l'exploitation : il reste à s'occuper des données.

- Bases relationnelles : rappel
- Le service managé
- Les moteurs
- Ce qu'AWS gère
- Ce qui reste à votre charge
- RDS ou base sur EC2

Les bases relationnelles, en rappel

Définition : base de données relationnelle

Base qui range les données dans des tables liées entre elles par des clés, selon un schéma défini à l'avance, et que l'on interroge en SQL.

Définition : transaction ACID

Suite d'opérations traitée comme un tout : entièrement appliquée ou pas du tout, laissant la base cohérente, isolée des autres transactions, et durable une fois validée.

Définition : SQL

Langage standard d'interrogation des bases relationnelles : sélection, jointure entre tables, agrégation, modification.

Ce pour quoi elles sont faites

- Des données structurées et liées : clients, commandes, lignes de commande, stock.
- Des requêtes variées, y compris celles que l'on n'avait pas prévues.
- Des règles d'intégrité garanties par la base : unicité, clés étrangères.
- Des transactions : une commande et sa mise à jour du stock réussissent ou échouent ensemble.

Ce qu'exige leur exploitation

- Installer, régler, corriger et mettre à jour le moteur.
- Sauvegarder, et savoir restaurer.
- Répliquer, basculer en cas de panne.
- Surveiller, agrandir le stockage.

Un travail continu, spécialisé, et sans valeur propre pour l'application.

Définition : Amazon RDS (Relational Database Service)

Service managé de bases de données relationnelles. Vous choisissez un moteur et une taille ; AWS installe, sauvegarde, met à jour, réplique et bascule.

Moteur	À savoir
PostgreSQL	Libre, très complet. Le choix par défaut de cette formation.
MySQL	Libre, très répandu dans le web.
MariaDB	Libre, issu de MySQL.
Oracle Database	Commercial ; licence comprise ou apportée.
Microsoft SQL Server	Commercial ; licence comprise.
IBM Db2	Commercial ; licence apportée.
Amazon Aurora	Moteur d'AWS, compatible MySQL ou PostgreSQL. Vu au chapitre « RDS, disponibilité et sauvegardes ».

Définition : instance de base de données

Environnement de base isolé dans RDS : un moteur qui tourne sur une machine d'une classe donnée, avec son stockage. Elle peut contenir plusieurs bases.

Définition : point de terminaison (endpoint)

Nom DNS par lequel les applications joignent l'instance. Il reste le même quand l'instance bascule ou change de machine.

C'est le vrai moteur. Une base RDS PostgreSQL est un PostgreSQL standard : mêmes clients, mêmes pilotes, même SQL. Migrer vers RDS ne demande aucune réécriture.

Schéma : qui fait quoi

Base installée sur EC2

Schéma, requêtes, index

Réglages du moteur

Sauvegardes, bascule

Mises à jour du moteur

Système d'exploitation

Matériel, réseau, bâtiment

Amazon RDS

Schéma, requêtes, index

Réglages du moteur

Sauvegardes, bascule

Mises à jour du moteur

Système d'exploitation

Matériel, réseau, bâtiment

DynamoDB

Modèle de données, requêtes

Réglages du moteur

Sauvegardes, bascule

Mises à jour du moteur

Système d'exploitation

Matériel, réseau, bâtiment

à votre charge

géré par AWS

Lecture. Plus le service est managé, moins il reste de couches à votre charge. Avec RDS, vous gardez ce qui fait la valeur de la base : son schéma, ses requêtes, ses réglages. Avec DynamoDB, il ne reste que le modèle de données.

C'est le modèle de responsabilité partagée du chapitre « Comprendre le cloud », appliqué aux bases de données : la ligne de partage monte à mesure que le service devient plus managé.

Ce qu'AWS gère, ce qui reste à votre charge

AWS s'en charge

Matériel, système d'exploitation, installation du moteur.

Correctifs et mises à jour mineures, dans une fenêtre que vous fixez.

Sauvegardes quotidiennes et journaux de transactions.

Réplication vers une zone de secours et bascule automatique.

Remplacement d'une machine défaillante.

Agrandissement du stockage.

Métriques et journaux vers CloudWatch.

Vous en restez responsable

Le schéma, les index, la qualité des requêtes.

Le dimensionnement : classe d'instance, stockage.

Le réseau : sous-réseaux, security group, pas d'accès public.

Les comptes de la base et leurs droits.

Le choix d'activer Multi-AZ, le chiffrement, la durée de rétention.

Les montées de version majeure, et leurs tests.

La vérification que les restaurations fonctionnent.

Managé ne veut pas dire sans réglage. RDS propose la haute disponibilité et les sauvegardes ; il ne les impose pas. Une base créée avec les options minimales n'a ni secours, ni chiffrement.

Une requête lente reste lente. RDS ne corrige ni un index manquant ni une jointure mal écrite : la performance applicative reste entièrement de votre ressort.

Limite	Conséquence
Aucun accès au système d'exploitation	Pas de SSH, pas d'agent à installer, pas de fichier à modifier.
Pas de compte superutilisateur complet	L'utilisateur principal a un rôle étendu (<code>rds_superuser</code>), mais pas tous les privilèges.
Extensions limitées à une liste	Seules les extensions prises en charge s'installent.
Réglages par groupe de paramètres	On ne modifie pas le fichier de configuration : on change un paramètre dans la console ou par l'API.
Versions imposées dans le temps	Une version en fin de vie doit être quittée, sous peine de mise à jour forcée ou de support payant.

Définition : groupe de paramètres

Ensemble nommé de réglages du moteur, appliqué à une ou plusieurs instances. Il remplace le fichier de configuration du moteur.

Quand ces limites gênent

- Logiciel qui exige un accès au système ou une extension non prise en charge.
- Version très ancienne, ou réglage interdit.

Deux issues : **RDS Custom**, qui ouvre l'accès au système pour Oracle et SQL Server, ou une base installée sur EC2.

Dans l'immense majorité des cas, ces limites ne se remarquent pas : elles sont le prix, très modeste, de l'exploitation déléguée.

	Amazon RDS	Base installée sur une instance EC2
Installation, correctifs	Automatiques.	À faire, et à refaire.
Sauvegardes	Intégrées, restauration à la seconde.	À concevoir, à tester, à surveiller.
Haute disponibilité	Une case à cocher.	Réplication et bascule à monter soi-même.
Accès au système	Non.	Complet.
Souplesse	Celle du service.	Totale : toute version, toute extension.
Prix de la machine	Plus élevé qu'une instance nue.	Celui de l'instance.
Coût réel	Tout compris.	Instance, plus le temps d'une personne compétente.

Règle. RDS par défaut. Une base sur EC2 seulement pour une contrainte précise qu'aucun service managé ne lève, en sachant que l'on prend alors toute l'exploitation à sa charge.

Rappel du chapitre « Élasticité ». Une base n'a pas sa place dans un Auto Scaling group ni sur une instance jetable : c'est un état à sortir des instances, et RDS est l'endroit prévu pour lui.

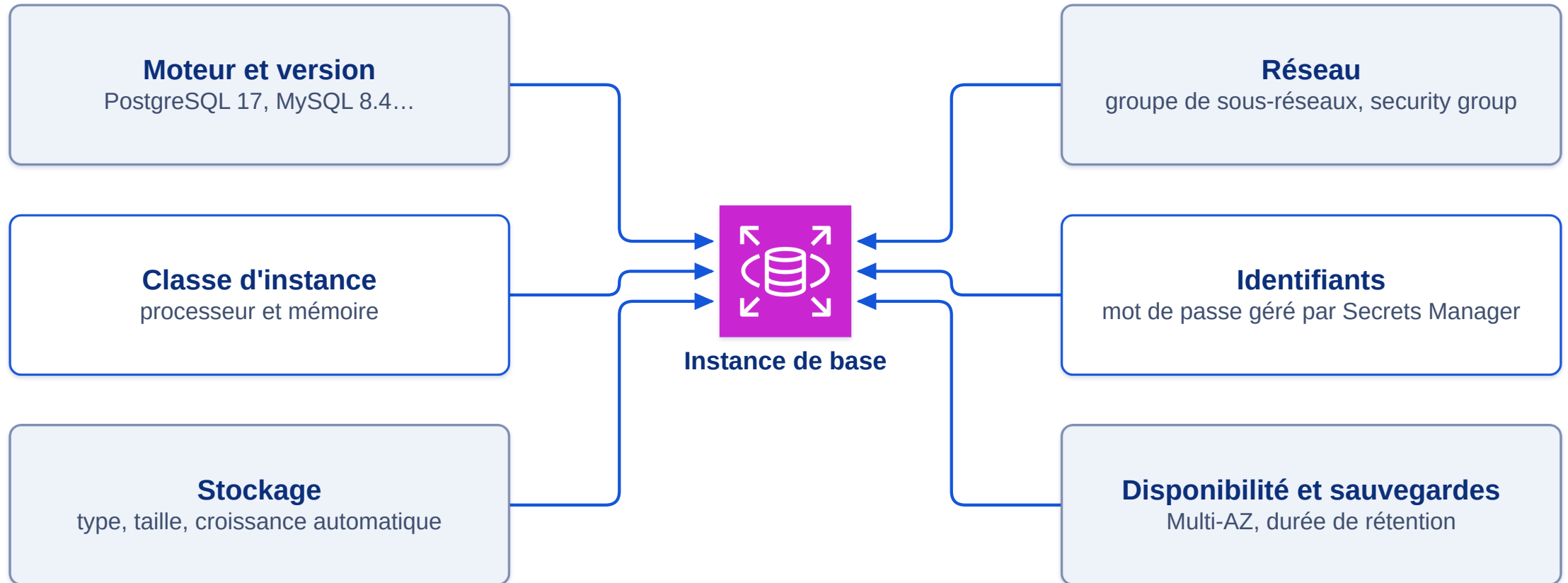
Erreur	Conséquence	Correction
Installer une base sur EC2 « parce que c'est moins cher »	Sauvegardes et bascule jamais vraiment mises en place.	RDS ; comparer le coût complet.
Croire que RDS optimise les requêtes	Base lente malgré une grosse instance.	Index, analyse des requêtes, Database Insights.
Garder toutes les options par défaut	Ni secours, ni protection contre la suppression.	Passer chaque option en revue.
Version du moteur laissée vieillir	Mise à jour forcée, ou frais de support étendu.	Planifier les montées de version.
Une instance par microservice, sans réflexion	Coût fixe multiplié.	Plusieurs bases sur une instance, ou Aurora Serverless.
Chercher à se connecter en SSH	Impossible.	Client SQL, par le point de terminaison.
Utiliser l'utilisateur principal dans l'application	Droits excessifs en cas de compromission.	Un compte applicatif aux droits limités.

RDS, créer et se connecter, partie 2

Créer une base et s'y connecter

Quatre choix déterminent une base RDS : sa puissance, son disque, sa place dans le réseau, et qui a le droit de lui parler.

- Les choix à la création
- La classe d'instance
- Le stockage
- Groupe de sous-réseaux et security group
- Chiffrement
- Création, connexion, coût



Lecture. Créer une base, c'est répondre à six questions. Certaines réponses se changent ensuite d'un clic, avec ou sans coupure : la classe, la taille du stockage, Multi-AZ. D'autres sont définitives ou coûteuses à revoir : le moteur, le chiffrement, le VPC. Les diapositives suivantes les prennent une à une.

Définition : classe d'instance de base de données

Type de machine sur lequel tourne le moteur. Elle fixe le processeur, la mémoire et le débit réseau de la base, et l'essentiel de son prix.

Famille	Profil	Usage
db.t4g, db.t3	À crédits de processeur.	Développement, tests, petites bases peu sollicitées.
db.m7g, db.m7i	Usage général : 4 Gio par vCPU.	Production courante.
db.r7g, db.r7i	Mémoire : 8 Gio par vCPU.	Bases volumineuses, nombreuses connexions.

Classe	vCPU	Mémoire	Prix horaire indicatif
db.t4g.micro	2	1 Gio	0,016 \$
db.t4g.medium	2	4 Gio	0,065 \$
db.m7g.large	2	8 Gio	0,17 \$
db.r7g.large	2	16 Gio	0,24 \$

La mémoire d'abord. Une base relationnelle est rapide tant que ses données actives tiennent en mémoire. Quand la métrique de mémoire disponible s'effondre et que les lectures sur disque montent, il faut une classe plus grande.

Changer de classe est possible à tout moment, avec une coupure de quelques minutes ; en Multi-AZ, elle se réduit à une bascule. Mêmes préfixes et mêmes règles de nommage que pour EC2, précédés de `db.`

Type	Caractéristique	Usage
gp3	SSD usage général : 3 000 IOPS de base, davantage au-delà de 400 Gio ; performance réglable.	Le choix par défaut.
io2	SSD à performance garantie, jusqu'à 256 000 IOPS.	Bases transactionnelles très sollicitées.
Magnétique	Ancienne génération.	À ne plus choisir.

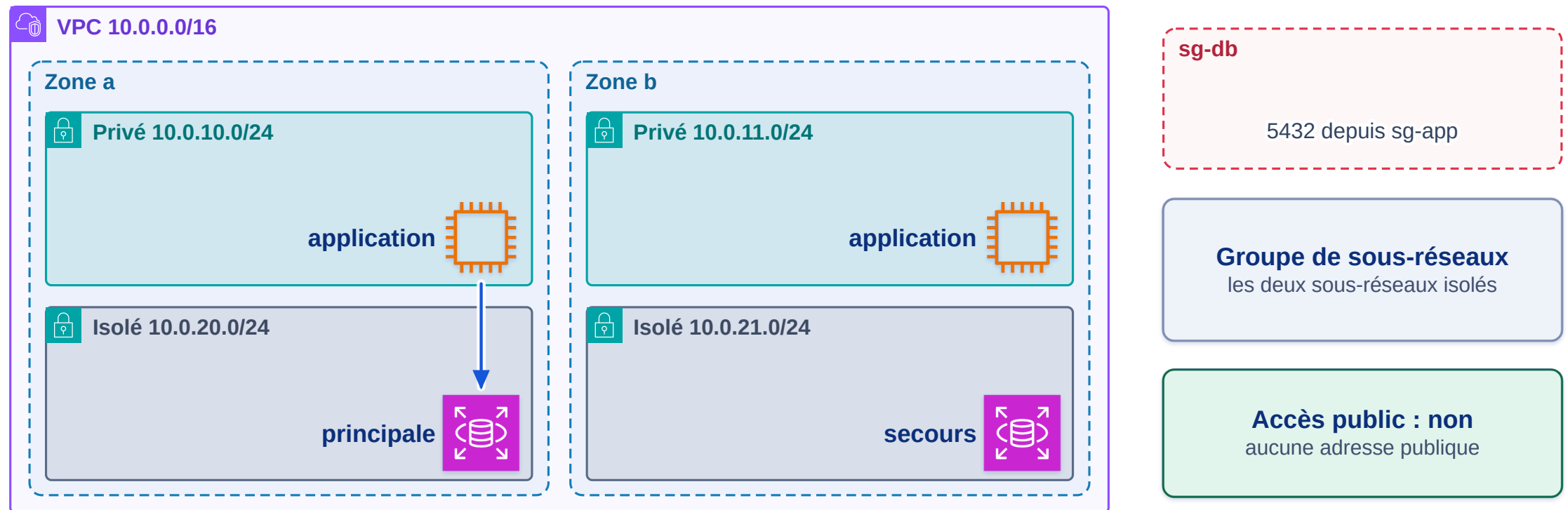
Stockage plein : base arrêtée. Sans espace libre, le moteur n'écrit plus et l'instance passe à l'état « storage-full ». Activer la mise à l'échelle automatique, et poser une alarme sur l'espace libre.

Définition : mise à l'échelle automatique du stockage

Option qui agrandit le stockage d'elle-même quand l'espace libre devient faible, jusqu'à un plafond que vous fixez.

Règle	Détail
Taille	De 20 Gio à 64 Tio selon le moteur.
Agrandir	À chaud, sans coupure.
Réduire	Impossible : il faut recréer la base.
Prix (gp3)	Environ 0,115 \$ par Go et par mois.

C'est de l'EBS. Le stockage d'une instance RDS repose sur des volumes EBS : mêmes types, mêmes notions d'IOPS et de débit que dans le chapitre « Volumes et images ». Ce sont eux que capturent les snapshots de la base.

**Définition : groupe de sous-réseaux de base de données (DB subnet group)**

Liste de sous-réseaux, dans au moins deux zones, où RDS peut placer l'instance et son secours. On y met les sous-réseaux isolés.

Lecture. La base vit dans les sous-réseaux isolés : aucune route vers Internet. Les applications des deux zones joignent l'instance principale ; celle de secours attend dans l'autre zone. Son security group n'accepte que le port du moteur, depuis le security group de l'application. C'est le dernier maillon du chaînage du chapitre « Filtrage et accès privés ».

Réglage	Valeur à retenir
VPC	Celui de l'application. Définitif.
Groupe de sous-réseaux	Les sous-réseaux isolés, deux zones au moins.
Accès public	Non. L'instance n'a alors aucune adresse publique.
Security group	Dédié à la base : port du moteur, depuis <code>sg-app</code> .
Port	5432 pour PostgreSQL, 3306 pour MySQL et MariaDB, 1433 pour SQL Server, 1521 pour Oracle.
Zone	Laissée au choix de RDS, ou imposée.

Le point de terminaison

- Forme : `nom.identifiant.région.rds.amazonaws.com`
- Il se résout en adresse **privée** depuis le VPC.
- L'application le lit dans sa configuration, jamais une adresse IP : celle-ci change à chaque bascule.

« **Publicly accessible : Yes** » donne une adresse publique à la base. Associé à un security group ouvert, c'est une base exposée à Internet : la cause de nombreuses fuites de données. Aucun cas courant ne le justifie.

Pour administrer depuis son poste : une redirection de port par Session Manager, vue plus loin. Jamais d'ouverture publique « le temps d'une intervention ».

Chiffrement	Détail
Au repos	Par une clé KMS : stockage, sauvegardes automatiques, snapshots, réplicas. Transparent pour l'application.
En transit	TLS entre l'application et la base, avec le certificat de l'autorité d'AWS.

Se décide à la création. Une instance créée sans chiffrement ne peut pas être chiffrée ensuite sur place. Il faut prendre un snapshot, le copier en le chiffrant, restaurer une nouvelle instance, puis basculer l'application.

Aucune raison de s'en passer : pas de surcoût, effet négligeable sur les performances. La console le propose activé.

Imposer TLS

- Côté base : paramètre `rds.force_ssl = 1` pour PostgreSQL, `require_secure_transport` pour MySQL. Déjà actif par défaut sur les versions récentes de PostgreSQL.
- Côté application : mode de connexion qui vérifie le certificat (`sslmode=verify-full`), avec le lot de certificats d'AWS.

\$ Télécharger le lot de certificats, puis se connecter en vérifiant

```
SITE=https://truststore.pki.rds.amazonaws.com
curl -sO $SITE/global/global-bundle.pem
```

```
psql "host=$ENDPOINT port=5432 dbname=shop user=shop_app \
sslmode=verify-full sslrootcert=global-bundle.pem"
```

awsRDS > Databases > Create database

Creation method

☒ Standard create

Engine

☒ PostgreSQL

Template

☒ Dev/Test

DB instance identifier

user7-shop-db

Credentials management

☒ Managed in AWS Secrets Manager

DB instance class

db.t4g.micro ▼

Storage

gp3, 20 GiB, autoscaling jusqu'à 100 GiB

DB subnet group

user7-db-subnets ▼

Public access

☒ No

VPC security group

user7-sg-db ▼

Encryption

☒ Enabled (aws/rds)

Backup retention

7 days ▼

Create database

Rubrique	Point d'attention
Template	« Production » active Multi-AZ et une classe plus grosse : regarder le coût estimé en bas de page.
Credentials	Mot de passe géré : rien à saisir, rien à retenir.
Connectivity	Vérifier VPC, groupe de sous-réseaux, accès public et security group : les valeurs par défaut désignent le VPC par défaut.
Additional configuration	Nom de la base initiale, rétention, protection contre la suppression.

Durée. La création prend de cinq à quinze minutes. L'instance passe par « creating », « backing-up », puis « available ».

« **Easy create** » décide de tout à votre place, y compris du réseau : à éviter dès qu'un VPC a été préparé.

Ligne de commande : créer une base

\$ Prérequis : sous-réseaux isolés ISO_A et ISO_B, security group SG_DB (5432 depuis sg-app) ; remplacer user7

1. Le groupe de sous-réseaux : les deux sous-réseaux isolés

```
aws rds create-db-subnet-group --db-subnet-group-name user7-db-subnets \  
  --db-subnet-group-description "Sous-reseaux isolees" --subnet-ids $ISO_A $ISO_B
```

2. L'instance : PostgreSQL, petite classe, chiffrée, privée, mot de passe géré par Secrets Manager

```
aws rds create-db-instance --db-instance-identifiant user7-shop-db \  
  --engine postgres --db-instance-class db.t4g.micro \  
  --allocated-storage 20 --storage-type gp3 --max-allocated-storage 100 --storage-encrypted \  
  --db-name shop --master-username shop_admin --manage-master-user-password \  
  --db-subnet-group-name user7-db-subnets --vpc-security-group-ids $SG_DB --no-publicly-accessible \  
  --backup-retention-period 7 --deletion-protection \  
  --tags Key=Owner,Value=user7 Key=Backup,Value=quotidien
```

3. Attendre qu'elle soit disponible (une dizaine de minutes), puis relever son point de terminaison

```
aws rds wait db-instance-available --db-instance-identifiant user7-shop-db  
ENDPOINT=$(aws rds describe-db-instances --db-instance-identifiant user7-shop-db \  
  --query 'DBInstances[0].Endpoint.Address' --output text)  
echo $ENDPOINT
```

4. État, classe, zone, Multi-AZ, chiffrement

```
aws rds describe-db-instances --db-instance-identifiant user7-shop-db --output table \  
  --query 'DBInstances[0].[DBInstanceStatus,DBInstanceClass,AvailabilityZone,MultiAZ,StorageEncrypted]'
```

Sans `--engine-version`, RDS choisit la version par défaut du moteur. En production, on la fixe explicitement pour que tous les environnements soient identiques.

Schéma : se connecter depuis son poste



Aucun port ouvert sur Internet, aucune adresse publique : la base reste inaccessible de l'extérieur.

\$ Prérequis : plugin Session Manager ; RELAIS = une instance gérée, autorisée par sg-db

```
aws ssm start-session --target $RELAIS \  
  --document-name AWS-StartPortForwardingSessionToRemoteHost \  
  --parameters "host=$ENDPOINT,portNumber=5432,localPortNumber=5432"
```

Dans un autre terminal

```
psql -h localhost -p 5432 -U shop_admin -d shop
```

Principe. Session Manager relie un port de votre poste à la base, à travers une instance du VPC que le security group de la base autorise. Aucune règle à ouvrir, aucun bastion public. Depuis une instance, on se connecte directement : `psql -h $ENDPOINT -U shop_admin -d shop`.

Ce que coûte une base RDS

Poste	Détail
Instance	À l'heure, selon la classe et le moteur. Doublé en Multi-AZ.
Stockage	Au gigaoctet provisionné. Doublé en Multi-AZ.
IOPS et débit supplémentaires	Si l'on dépasse la base de gp3, ou en io2.
Sauvegardes	Gratuites jusqu'à la taille de la base ; au-delà, au gigaoctet.
Transfert	Gratuit dans la zone ; facturé entre zones et vers Internet.
Secret géré	0,40 \$ par mois.

Exemple mensuel (PostgreSQL)	Ordre de grandeur
db.t4g.micro, 20 Gio, une zone	14 \$
db.t4g.medium, 100 Gio, une zone	59 \$
db.m7g.large, 100 Gio, Multi-AZ	270 \$

Réduire la facture. Engagement d'un ou trois ans (instances réservées), processeurs Graviton, classe ajustée aux mesures, et arrêt des bases de test hors des heures de travail.

Arrêter une base. Une instance arrêtée ne coûte plus que son stockage, mais elle **redémarre toute seule au bout de 7 jours**. Pour une base de test durablement inutile : snapshot final, puis suppression.

Erreur	Conséquence	Correction
Accès public activé	Base joignable depuis Internet.	<code>--no-publicly-accessible</code> , sous-réseaux isolés.
Security group de la base ouvert à une plage, ou à 0.0.0.0/0	Tout le VPC, ou tout Internet, atteint le port.	Source : le security group de l'application.
Groupe de sous-réseaux sur une seule zone	Création refusée, ou Multi-AZ impossible.	Deux zones au moins.
Base créée sans chiffrement	Recréation nécessaire pour chiffrer.	<code>--storage-encrypted</code> dès la création.
Pas de mise à l'échelle du stockage	Base arrêtée, disque plein.	<code>--max-allocated-storage</code> , alarme sur l'espace libre.
Classe à crédits en production chargée	Ralentissements quand les crédits s'épuisent.	Famille m ou r.
Adresse IP de la base dans la configuration	Panne à la première bascule.	Le point de terminaison.
Connexion sans TLS	Identifiants et données en clair sur le réseau.	Imposer TLS, vérifier le certificat.
Délai dépassé à la connexion	Security group, ou mauvais sous-réseau.	Méthode du chapitre « Dépannage réseau » ; Reachability Analyzer.

RDS, créer et se connecter, partie 3

Le mot de passe géré par Secrets Manager

Le mot de passe de la base n'a pas à être connu de qui que ce soit : RDS le crée, le range et le change tout seul.

- Le problème des mots de passe de base
- Secrets Manager
- Le mot de passe principal géré
- Lecture par l'application
- Autres façons de s'authentifier
- Synthèse : RDS, créer et se connecter

Pratique courante	Ce qui se passe
Mot de passe dans le code ou un fichier de configuration	Il finit dans le dépôt Git, les sauvegardes, les journaux.
Mot de passe dans une variable d'environnement en clair	Visible dans la console, la définition de tâche, l'historique.
Mot de passe dans le user data	Lisible par tout processus de l'instance et par l'API.
Même mot de passe pour tous les environnements	Une fuite en test ouvre la production.
Mot de passe jamais changé	Connu de tous ceux qui sont passés par l'équipe.

Ce qu'il faudrait

- Un mot de passe long et aléatoire, que personne ne connaît.
- Rangé à un seul endroit, chiffré.
- Lisible seulement par les identités autorisées, avec une trace de chaque lecture.
- Changer régulièrement, sans intervention ni coupure.

Définition : secret

Information sensible dont dépend un accès : mot de passe, clé d'API, jeton, certificat. Sa divulgation donne les droits de son détenteur.

Pour le mot de passe principal d'une base RDS, ces quatre exigences tiennent en une option à la création.

Définition : AWS Secrets Manager

Service qui conserve des secrets chiffrés, en contrôle l'accès par IAM, trace chaque lecture, et sait les renouveler automatiquement.

Fonction	Détail
Stockage chiffré	Chaque secret est chiffré par une clé KMS.
Contrôle d'accès	Politiques IAM, et politique propre au secret.
Trace	Chaque lecture figure dans CloudTrail.
Rotation	Renouvellement programmé du secret, et de sa valeur dans la base.
Versions	La valeur courante et la précédente coexistent le temps d'une rotation.
Réplication	Copie du secret dans d'autres régions.

Un secret de base de données

Un document JSON, pas seulement un mot de passe :

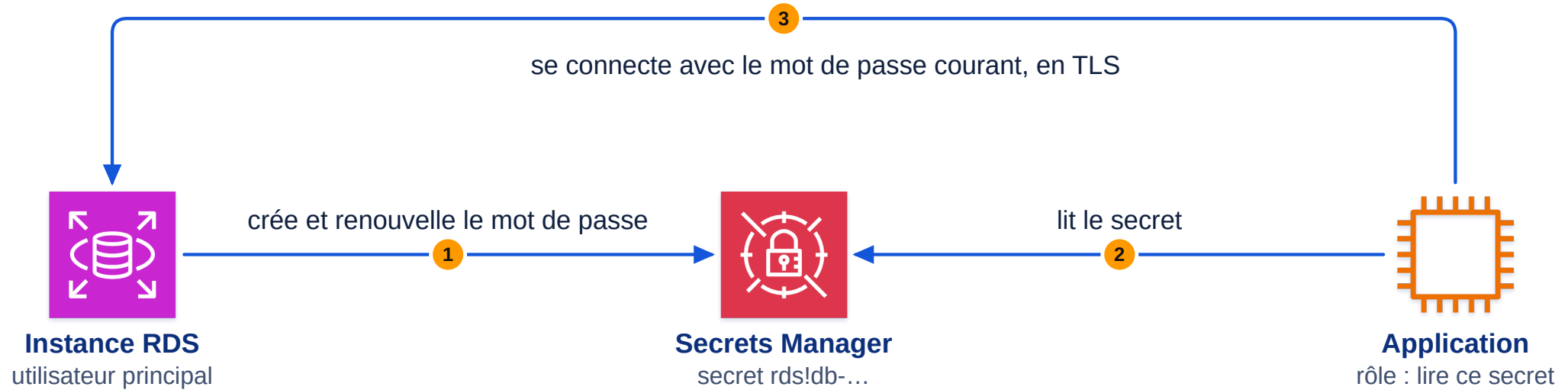
- username , password
- host , port , dbname , engine

L'application y trouve tout ce qu'il faut pour se connecter.

Coût	Tarif indicatif
Par secret	0,40 \$ par mois
Par 10 000 appels	0,05 \$

Ce service est repris en détail dans le chapitre « Choisir sa base, protéger ses secrets », avec Parameter Store et KMS.

Schéma : le mot de passe principal géré



1. À la création, RDS génère le mot de passe de l'utilisateur principal, le range dans un secret, et le renouvelle ensuite tous les 7 jours par défaut.

2. L'application, grâce à son rôle, lit le secret quand elle en a besoin. Personne n'a jamais vu le mot de passe.

3. Elle se connecte à la base avec la valeur courante.

Mettre en œuvre le mot de passe géré

\$ Prérequis : l'instance user7-shop-db a été créée avec --manage-master-user-password

1. L'ARN du secret créé par RDS

```
SECRET_ARN=$(aws rds describe-db-instances --db-instance-identifier user7-shop-db \
--query 'DBInstances[0].MasterUserSecret.SecretArn' --output text)
```

2. Lire le secret (demande le droit secretsmanager:GetSecretValue, et kms:Decrypt si la clé est gérée par vous)

```
aws secretsmanager get-secret-value --secret-id $SECRET_ARN --query SecretString --output text
```

3. Se connecter sans jamais afficher le mot de passe (jq requis)

```
export PGPASSWORD=$(aws secretsmanager get-secret-value --secret-id $SECRET_ARN \
--query SecretString --output text | jq -r .password)
psql -h $ENDPOINT -U shop_admin -d shop
```

4. Passer une instance existante au mot de passe géré

```
aws rds modify-db-instance --db-instance-identifier user7-shop-db --manage-master-user-password --apply-immediately
```

5. Quand la prochaine rotation aura-t-elle lieu ?

```
aws secretsmanager describe-secret --secret-id $SECRET_ARN --query '[RotationEnabled,NextRotationDate]'
```

Droit à donner à l'application. Une politique sur son rôle :
secretsmanager:GetSecretValue, limitée à l'ARN de ce secret. Rien d'autre.

Le secret appartient à RDS. On ne le modifie pas et on ne le supprime pas à la main : il disparaît avec l'instance, ou quand on désactive l'option.

py Python (boto3, pycopg) : sur une instance ou dans un conteneur doté d'un rôle

```
import json, boto3, pycopg

def connexion():
    sm = boto3.client("secretsmanager",
                      region_name="eu-west-3")
    brut = sm.get_secret_value(
        SecretId="rds!db-1a2b3c4d")["SecretString"]
    s = json.loads(brut)
    return pycopg.connect(
        host="shop-db.c1x2.eu-west-3.rds.amazonaws.com",
        port=5432, dbname="shop",
        user=s["username"], password=s["password"],
        sslmode="verify-full",
        sslrootcert="global-bundle.pem")

# En cas d'échec d'authentification :
# relire le secret, puis réessayer une fois
```

Vivre avec la rotation

- Lire le secret au démarrage et le garder en mémoire quelques minutes, pas plus : un appel par requête SQL coûterait cher et ralentirait tout.
- Si la base refuse l'authentification, relire le secret et réessayer : le mot de passe vient sans doute de changer.
- Les connexions déjà ouvertes survivent à une rotation ; seules les nouvelles demandent le nouveau mot de passe.

Bibliothèques de cache. AWS fournit, pour plusieurs langages, un client qui garde les secrets en mémoire et les rafraîchit : inutile de l'écrire soi-même.

Un compte par usage. L'utilisateur principal sert à administrer. L'application utilise un compte aux droits limités, dont le secret est lui aussi dans Secrets Manager, avec sa propre rotation.

Définition : authentification IAM à la base

Mode dans lequel l'application se connecte avec un jeton temporaire, valable 15 minutes, obtenu grâce à son rôle IAM, au lieu d'un mot de passe.

Définition : RDS Proxy

Service placé entre les applications et la base, qui mutualise les connexions et les maintient pendant une bascule. Il lit lui-même les identifiants dans Secrets Manager.

	Mot de passe géré	Authentification IAM
Secret à lire	Oui, dans Secrets Manager.	Aucun.
Durée de vie	Jours.	15 minutes.
Moteurs	Tous.	PostgreSQL, MySQL, MariaDB.
Limite	Coût par secret.	Nombre de nouvelles connexions par seconde limité.

Quand ajouter RDS Proxy

- Très nombreuses connexions courtes : fonctions Lambda, flotte qui grossit vite.
- Base saturée par le nombre de connexions plus que par la charge.
- Besoin de bascules presque invisibles pour l'application.

À retenir pour la suite. Une fonction Lambda qui parle à une base relationnelle passe presque toujours par RDS Proxy : sans lui, chaque exécution ouvre sa propre connexion.

Ce qu'il faut retenir

- RDS exploite la base : installation, correctifs, sauvegardes, bascule. Schéma, requêtes et dimensionnement restent à vous.
- C'est le moteur standard : PostgreSQL, MySQL, MariaDB, Oracle, SQL Server, Db2.
- La classe d'instance fixe processeur et mémoire ; le stockage gp3 s'agrandit, ne se réduit pas.
- La base vit dans les sous-réseaux isolés, sans accès public, derrière un security group qui n'accepte que l'application.
- Le chiffrement se choisit à la création.
- Le mot de passe principal est géré par Secrets Manager : créé, rangé, renouvelé.
- L'application lit le secret grâce à son rôle.

1. Peut-on se connecter en SSH à une instance RDS ?

2. Que contient un groupe de sous-réseaux de base de données ?

3. Quelle source dans le security group de la base ?

4. Peut-on réduire le stockage d'une instance ?

5. Peut-on chiffrer une instance existante ?

6. Qu'arrive-t-il à une instance arrêtée depuis 7 jours ?

7. Quelle option fait gérer le mot de passe par Secrets Manager ?

Cliquer sur une question pour afficher la réponse.

RDS, disponibilité et sauvegardes, partie 1

Disponibilité : Multi-AZ et réplicas de lecture

Deux mécanismes de réplication, souvent confondus : l'un fait survivre la base à une panne, l'autre la soulage de ses lectures.

- Les pannes à couvrir
- Le déploiement Multi-AZ
- La bascule
- Les réplicas de lecture
- Multi-AZ ou réplica
- Mise en œuvre

Événement	Conséquence sans protection	Réponse
Panne de la machine hôte	Base indisponible le temps du remplacement.	Multi-AZ.
Panne d'une zone de disponibilité	Base indisponible, peut-être longtemps.	Multi-AZ.
Maintenance, changement de classe	Coupure de plusieurs minutes.	Multi-AZ : réduite à une bascule.
Base saturée par les lectures	Lenteur générale.	Réplicas de lecture, cache.
Suppression ou corruption de données	Données perdues.	Sauvegardes, restauration à un instant.
Indisponibilité de la région	Tout est arrêté.	Réplica ou sauvegardes dans une autre région.

Définition : haute disponibilité

Capacité d'un système à rester utilisable malgré la panne d'un de ses composants, grâce à la redondance et à une bascule automatique.

La réplication ne protège pas des erreurs. Un `DELETE` sans clause `WHERE` est répliqué instantanément vers le secours et vers tous les réplicas. Contre l'erreur humaine, il n'y a que les sauvegardes.

Définition : déploiement Multi-AZ

Configuration dans laquelle RDS maintient, dans une autre zone de disponibilité, une copie synchrone de la base, et bascule sur elle automatiquement en cas de panne.

Propriété	Détail
Réplication	Synchrone : une écriture n'est validée que lorsqu'elle est enregistrée dans les deux zones.
Perte de données	Aucune à la bascule.
Instance de secours	Invisible : ni lisible, ni joignable.
Point de terminaison	Unique, inchangé après bascule.
Activation	À la création, ou plus tard, sans coupure.
Coût	Le double : deux instances, deux stockages.

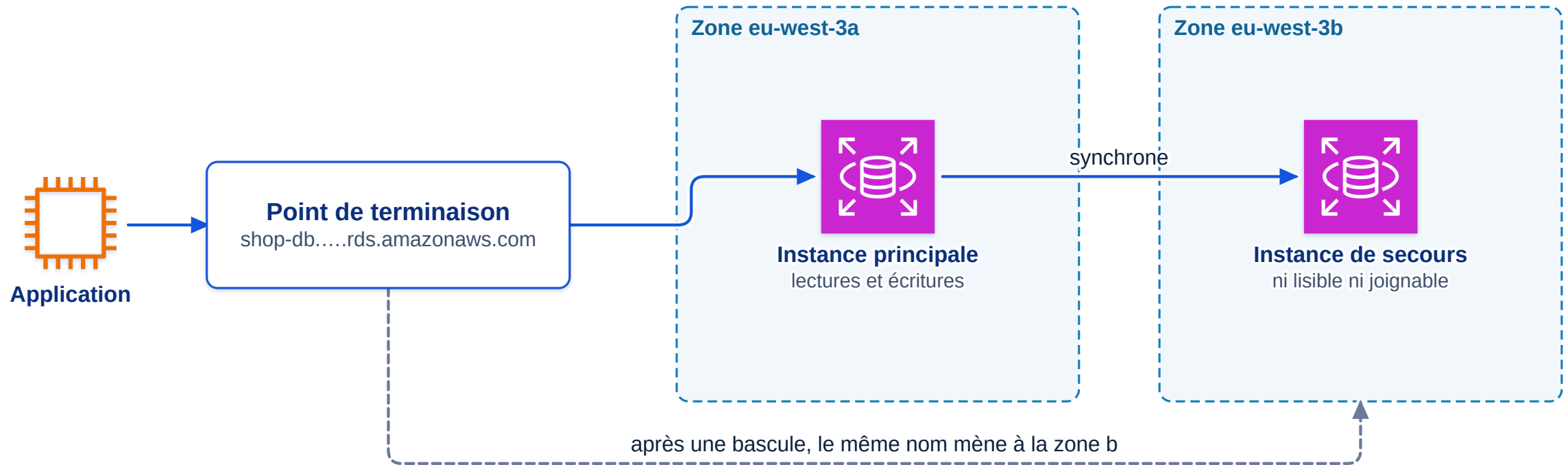
Ce qu'il apporte

- La base survit à la panne d'une machine ou d'une zone.
- Les maintenances se font d'abord sur le secours, puis par bascule : la coupure tombe d'une dizaine de minutes à une ou deux.
- Les sauvegardes sont prises sur le secours, sans gêner la base principale.

Ce qu'il n'apporte pas. Aucune capacité de lecture supplémentaire : le secours ne sert à rien tant qu'il n'y a pas de panne. Et aucune protection contre l'erreur de manipulation.

Règle. Multi-AZ pour toute base de production. Une zone suffit en développement et en test.

Schéma : le déploiement Multi-AZ

**Définition : réplication synchrone**

Réplication dans laquelle l'écriture n'est confirmée à l'application qu'une fois enregistrée sur la copie. Aucune donnée validée ne peut manquer sur le secours.

Lecture. L'application ne connaît qu'un nom. En temps normal, il désigne l'instance de la zone a ; après une bascule, celle de la zone b. C'est RDS qui met à jour l'enregistrement DNS, dont la durée de vie est de quelques secondes.

Schéma : le déroulement d'une bascule



Rien à faire, rien à changer dans la configuration : le nom de la base reste le même.

L'application doit seulement savoir se reconnecter après une coupure.

Ce qui déclenche une bascule

Panne de l'instance, de son stockage ou de son réseau.

Indisponibilité de la zone.

Changement de classe, mise à jour du système.

Redémarrage demandé « avec bascule », pour s'exercer.

Côté application

Les connexions ouvertes sont coupées. Il faut : se reconnecter avec de nouvelles tentatives espacées, ne pas garder l'adresse IP en cache au-delà de quelques secondes, et accepter qu'une transaction en cours échoue et soit rejouée.

S'exercer. Une bascule jamais essayée révèle le jour de la panne que l'application ne sait pas se reconnecter.

Deux formes de Multi-AZ

	Instance Multi-AZ	Cluster Multi-AZ
Composition	Une instance principale, une de secours.	Une instance d'écriture, deux de secours.
Zones	Deux.	Trois.
Secours lisible	Non.	Oui : les deux servent les lectures.
Durée de bascule	Une à deux minutes.	En général moins de 35 secondes.
Écritures	Validées sur le secours.	Validées sur au moins un des deux secours.
Moteurs	Tous.	MySQL et PostgreSQL.
Points de terminaison	Un.	Un pour l'écriture, un pour la lecture.
Coût	Deux instances.	Trois instances.

Choisir. L'instance Multi-AZ suffit à la plupart des applications. Le cluster se justifie quand la bascule doit être plus rapide, ou quand on veut que les machines de secours servent aussi à lire.

Et Aurora ? Il répond au même besoin par une autre architecture, vue en fin de chapitre : un stockage partagé entre les zones, des bascules encore plus rapides.

Définition : réplica de lecture

Copie de la base, tenue à jour de façon asynchrone, sur laquelle on ne fait que lire. Elle a son propre point de terminaison, et peut se trouver dans une autre zone ou une autre région.

Propriété	Détail
Réplication	Asynchrone : le réplica suit la base principale avec un léger retard.
Usage	Lecture seule.
Nombre	Jusqu'à 15 pour MySQL, MariaDB et PostgreSQL.
Emplacement	Même zone, autre zone, autre région.
Bascule	Aucune automatique : la promotion est une décision manuelle.
Classe	Libre : un réplica peut être plus petit ou plus gros que la base principale.

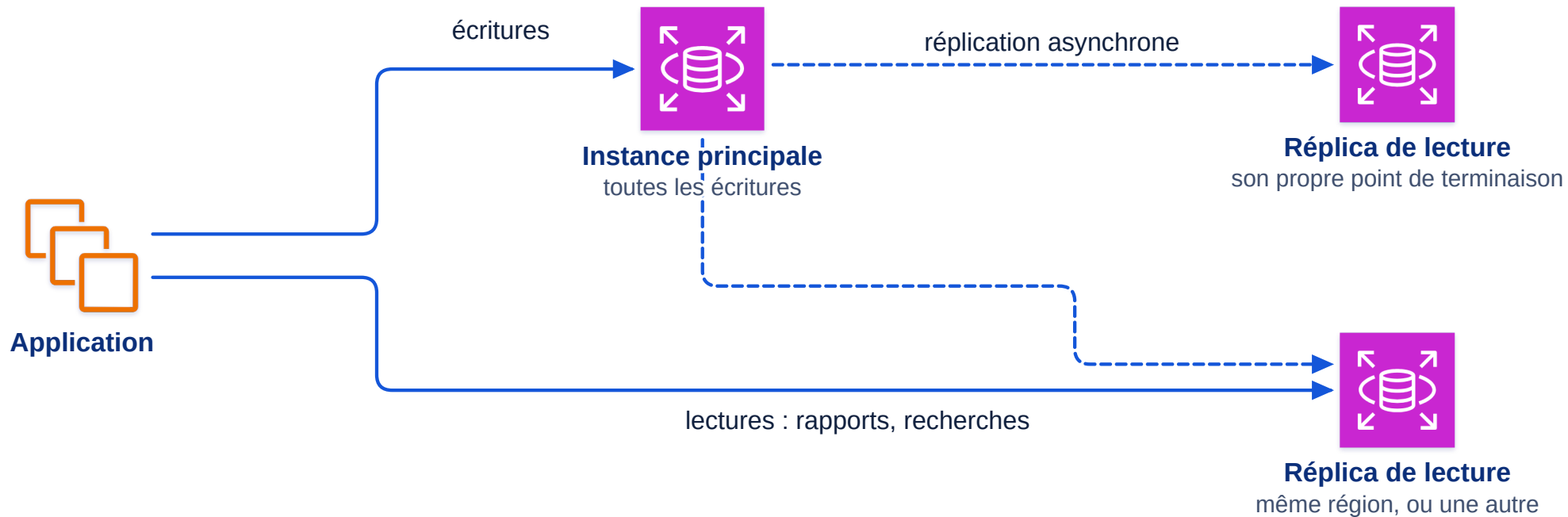
À quoi ils servent

- Décharger la base principale des lectures lourdes : rapports, tableaux de bord, recherches, exports.
- Servir des lecteurs éloignés, depuis un réplica dans leur région.
- Disposer d'une copie dans une autre région, à promouvoir en cas de sinistre.
- Préparer une migration ou une montée de version.

Définition : retard de réplication (replica lag)

Délai entre une écriture sur la base principale et son apparition sur le réplica. Quelques millisecondes en temps normal, davantage sous forte charge d'écriture.

Schéma : les réplicas de lecture



Lire un réplica, c'est lire un passé récent. Une donnée écrite à l'instant peut ne pas encore s'y trouver. On y envoie ce qui tolère ce décalage : statistiques, listes, recherches. Ce qu'un utilisateur vient d'enregistrer se relit sur la base principale.

C'est l'application qui aiguille. RDS ne répartit pas les requêtes : le code doit utiliser deux connexions, une pour écrire, une pour lire. Certaines bibliothèques et RDS Proxy facilitent ce partage.

Disponibilité

Multi-AZ ou réplica de lecture

	Multi-AZ	Réplica de lecture
But	Disponibilité : survivre à une panne.	Performance : répartir les lectures.
Réplication	Synchrone.	Asynchrone.
Copie utilisable	Non (instance Multi-AZ).	Oui, en lecture.
Bascule	Automatique.	Manuelle : promotion.
Point de terminaison	Le même.	Un par réplica.
Perte de données possible	Aucune.	Les dernières écritures non répliquées.
Autre région	Non.	Oui.
Changement dans l'application	Aucun.	Aiguiller les lectures.

Ils se combinent. Une base de production sérieuse : Multi-AZ pour la disponibilité, un ou deux réplicas pour les lectures lourdes, éventuellement un réplica dans une autre région pour la reprise après sinistre. Un réplica peut lui-même être Multi-AZ.

Moyen de s'en souvenir. Multi-AZ répond à « et si ça tombe ? ». Le réplica répond à « et si ça rame ? ». Aucun des deux ne répond à « et si on efface tout ? ».

\$ Prérequis : l'instance user7-shop-db existe, sauvegardes automatiques activées

1. Passer une instance en Multi-AZ (sans coupure ; quelques minutes de synchronisation)

```
aws rds modify-db-instance --db-instance-identifiant user7-shop-db --multi-az --apply-immediately
```

2. Où sont l'instance principale et son secours ?

```
aws rds describe-db-instances --db-instance-identifiant user7-shop-db \
  --query 'DBInstances[0].[MultiAZ,AvailabilityZone,SecondaryAvailabilityZone]' --output text
```

3. S'exercer : provoquer une bascule, puis relire la zone

```
aws rds reboot-db-instance --db-instance-identifiant user7-shop-db --force-failover
```

4. Créer un réplica de lecture, puis relever son point de terminaison

```
aws rds create-db-instance-read-replica --db-instance-identifiant user7-shop-db-lecture \
  --source-db-instance-identifiant user7-shop-db --db-instance-class db.t4g.micro
```

```
aws rds describe-db-instances --db-instance-identifiant user7-shop-db-lecture \
  --query 'DBInstances[0].Endpoint.Address' --output text
```

5. Promouvoir un réplica : il devient une base indépendante, en lecture et écriture (irréversible)

```
aws rds promote-read-replica --db-instance-identifiant user7-shop-db-lecture
```

6. Événements récents de l'instance : bascules, redémarrages, sauvegardes

```
aws rds describe-events --source-identifiant user7-shop-db --source-type db-instance --duration 1440 \
  --query 'Events[].[Date,Message]' --output text
```

Suivre le retard d'un réplica. Métrique CloudWatch `ReplicaLag`, en secondes : y poser une alarme. Un retard qui grandit sans cesse signale un réplica trop petit pour suivre les écritures.

Erreur	Conséquence	Correction
Production sur une seule zone	Arrêt à la première panne ou maintenance.	Multi-AZ.
Prendre un réplica pour un secours	Aucune bascule automatique ; données récentes manquantes.	Multi-AZ pour la disponibilité.
Attendre de Multi-AZ un gain en lecture	Aucun : le secours n'est pas lisible.	Réplica, ou cluster Multi-AZ.
Relire sur un réplica ce que l'on vient d'écrire	Donnée « disparue » par moments.	Relire sur la base principale.
Application qui ne se reconnecte pas	Bascule réussie, service toujours en panne.	Nouvelles tentatives ; tester une bascule.
Adresse IP gardée en cache longtemps	L'application insiste sur l'ancienne instance.	Respecter la durée de vie du DNS.
Croire la réplication suffisante contre l'erreur	Suppression répliquée partout.	Sauvegardes et restauration à un instant.
Réplica trop petit	Retard qui s'accumule.	Même classe que la base principale, ou plus.

RDS, disponibilité et sauvegardes, partie 2

Sauvegardes et restauration

Revenir à l'état de la base tel qu'il était avant l'erreur, à la seconde près.

- Trois mécanismes
- Les sauvegardes automatiques
- La restauration à un instant donné
- Les snapshots manuels
- Restaurer : une nouvelle instance
- Protection, maintenance, surveillance

Sauvegardes

Trois mécanismes de sauvegarde

	Sauvegardes automatiques	Snapshot manuel	AWS Backup
Déclenchement	Chaque jour, dans une fenêtre choisie, plus les journaux en continu.	À la demande.	Selon un plan commun à plusieurs services.
Conservation	De 1 à 35 jours.	Jusqu'à ce qu'on le supprime.	Celle du plan : jours, mois, années.
Restauration	À n'importe quelle seconde de la période.	À l'instant du snapshot.	Au point de récupération, ou à un instant si la sauvegarde continue est activée.
À la suppression de l'instance	Supprimées, sauf demande contraire.	Conservé.	Conservé dans le coffre.
Usage	Le filet quotidien.	Avant une opération risquée ; archive durable.	Politique d'entreprise, copie dans un autre compte.

Les trois reposent sur la même technique : des snapshots du stockage de l'instance, incrémentaux, conservés dans S3 par AWS. Seuls changent qui les déclenche et combien de temps ils vivent.

Combinaison habituelle. Sauvegardes automatiques sur 7 à 35 jours pour l'erreur du quotidien ; AWS Backup pour la conservation longue et la copie hors du compte ; un snapshot manuel avant chaque migration.

Sauvegardes

Les sauvegardes automatiques

Définition : sauvegarde automatique RDS

Sauvegarde quotidienne de toute l'instance, complétée par l'archivage continu des journaux de transactions, conservée pendant une durée de rétention comprise entre 1 et 35 jours.

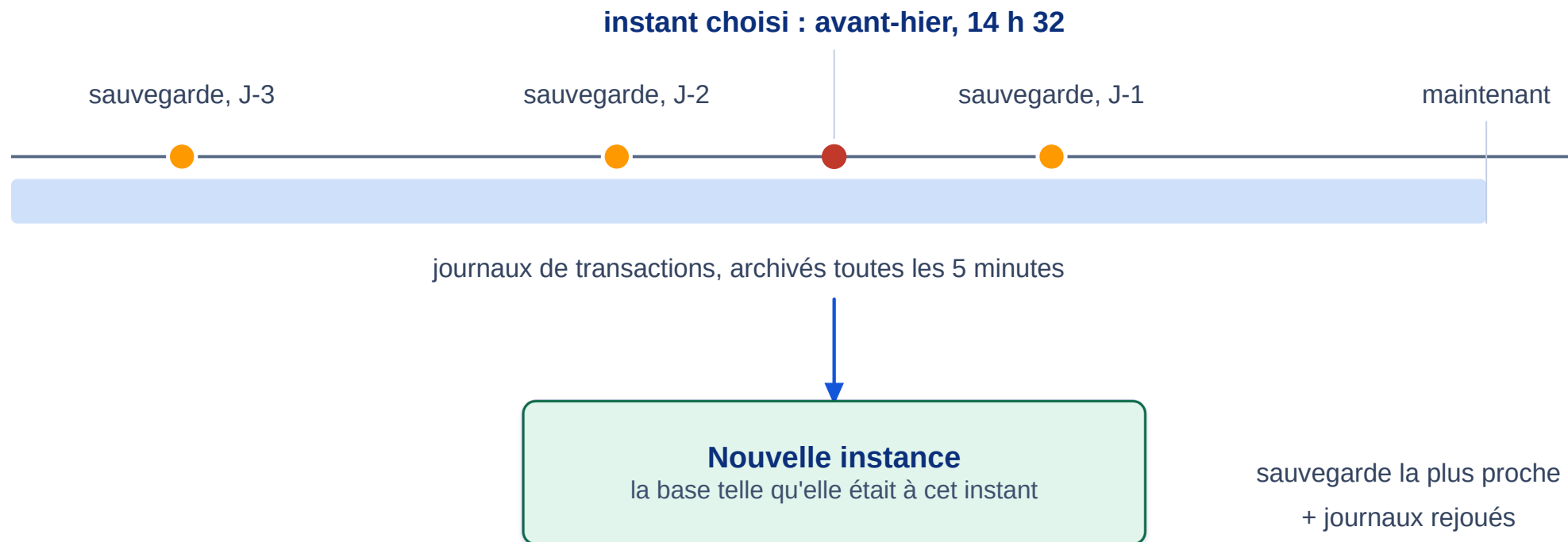
Réglage	Détail
Durée de rétention	De 1 à 35 jours. Zéro désactive les sauvegardes, et avec elles la restauration à un instant et les réplicas.
Fenêtre de sauvegarde	Plage horaire quotidienne, à placer aux heures creuses.
Journaux de transactions	Envoyés vers S3 toutes les 5 minutes.
Copie dans une autre région	Réplication automatique des sauvegardes, en option.

À savoir

- La première sauvegarde est complète ; les suivantes ne stockent que les blocs modifiés.
- Sur une instance Multi-AZ, la sauvegarde est prise sur le secours : aucun effet sur la production.
- Sur une instance à une zone, une brève suspension des écritures est possible au début de la sauvegarde.
- Le stockage des sauvegardes est gratuit jusqu'à la taille de la base.

Valeur par défaut. 7 jours depuis la console, mais 1 jour seulement par la ligne de commande et l'API : toujours préciser la rétention dans un script.

Schéma : la restauration à un instant donné

**Définition : restauration à un instant donné (PITR)**

Reconstitution de la base telle qu'elle était à une seconde précise de la période de rétention : RDS part de la sauvegarde précédente et rejoue les journaux de transactions jusqu'à l'instant voulu.

Le cas d'usage type. À 14 h 35, quelqu'un supprime la table des commandes. On restaure à 14 h 34 : tout ce qui précède l'erreur est là. Le dernier instant restaurable se situe en général moins de 5 minutes avant le présent.

Les snapshots manuels

Définition : snapshot de base de données

Copie de l'état complet d'une instance RDS à un instant donné, déclenchée à la demande, et conservée jusqu'à sa suppression explicite.

Opération	Détail
Copier	Vers une autre région, en chiffrant au passage si besoin.
Partager	Avec un autre compte ; clé gérée par le client requise si chiffré.
Restaurer	Vers une nouvelle instance, d'une classe ou d'un stockage différents.
Exporter	Vers S3, au format Parquet, pour analyse.

Quand en prendre un

- Avant une migration de schéma ou une montée de version majeure.
- Avant de supprimer une instance : le **snapshot final**.
- Pour cloner la production vers un environnement de test.
- Pour garder un état de référence au-delà de 35 jours.

Définition : snapshot final

Snapshot proposé au moment de supprimer une instance. Sans lui, et sans sauvegardes conservées, la base est perdue définitivement.

Ils s'accablent. Un snapshot manuel n'expire jamais et se paie au gigaoctet. Les nommer avec une date, et les passer en revue.

Schéma : une restauration crée une nouvelle instance



Une restauration n'écrase jamais l'instance d'origine.

C'est une sécurité, et une étape de plus : il faut rediriger l'application.

Ce que la nouvelle instance ne reprend pas d'office

Le security group, le groupe de paramètres, Multi-AZ, la protection contre la suppression : à préciser à la restauration, sinon ce sont les valeurs par défaut. Les sauvegardes automatiques, elles, repartent de zéro.

Rediriger l'application. Deux façons : modifier le nom de base dans sa configuration, ou renommer l'ancienne instance puis donner son nom à la nouvelle, ce qui reprend le même point de terminaison. Une zone privée Route 53 avec un nom stable (`db.shop.interne`) simplifie encore l'opération.

\$ Prérequis : l'instance user7-shop-db existe ; SG_DB et le groupe de sous-réseaux user7-db-subnets définis

1. Période restaurable : du plus ancien instant au plus récent

```
aws rds describe-db-instance-automated-backups --db-instance-identifiant user7-shop-db \
--query 'DBInstanceAutomatedBackups[0].RestoreWindow' --output table
```

2. Snapshot manuel avant une opération risquée

```
aws rds create-db-snapshot --db-instance-identifiant user7-shop-db \
--db-snapshot-identifiant user7-shop-db-avant-migration
aws rds wait db-snapshot-available --db-snapshot-identifiant user7-shop-db-avant-migration
```

3. Restaurer à un instant précis (heure UTC), vers une NOUVELLE instance

```
aws rds restore-db-instance-to-point-in-time \
--source-db-instance-identifiant user7-shop-db --target-db-instance-identifiant user7-shop-db-restaure \
--restore-time 2026-10-05T12:34:00Z \
--db-subnet-group-name user7-db-subnets --vpc-security-group-ids $SG_DB --no-publicly-accessible
```

4. Ou restaurer depuis le snapshot

```
aws rds restore-db-instance-from-db-snapshot --db-instance-identifiant user7-shop-db-clone \
--db-snapshot-identifiant user7-shop-db-avant-migration \
--db-subnet-group-name user7-db-subnets --vpc-security-group-ids $SG_DB --no-publicly-accessible
```

5. Supprimer une instance en gardant un snapshot final (lever d'abord la protection contre la suppression)

```
aws rds modify-db-instance --db-instance-identifiant user7-shop-db-clone --no-deletion-protection --apply-immediately
aws rds delete-db-instance --db-instance-identifiant user7-shop-db-clone \
--final-db-snapshot-identifiant user7-shop-db-clone-final
```

Durée d'une restauration. De quelques minutes à plusieurs heures selon la taille de la base et la quantité de journaux à rejouer : c'est elle qui fixe le RTO réel. Il faut l'avoir mesurée avant d'en avoir besoin.

Sauvegardes

Protéger la base contre la suppression

Protection	Effet
Protection contre la suppression	L'instance ne peut pas être supprimée tant que l'option est active. À activer sur toute base qui compte.
Snapshot final	Garde une copie au moment de la suppression.
Conservation des sauvegardes automatiques	Option qui les garde après la suppression de l'instance, jusqu'à la fin de leur rétention.
AWS Backup avec Vault Lock	Sauvegardes que personne ne peut effacer avant l'échéance.
Copie dans une autre région ou un autre compte	Survit à la perte de la région, ou à la compromission du compte.
Droits IAM	Réserver <code>rds:DeleteDBInstance</code> et la suppression de snapshots à très peu d'identités.

Le scénario à éviter

Un script de nettoyage, ou un `terraform destroy` lancé dans le mauvais environnement, supprime l'instance de production sans snapshot final : les sauvegardes automatiques partent avec elle.

Protection contre la suppression et sauvegardes hors de l'instance rendent ce scénario impossible.

Règle. Aucune base de production sans protection contre la suppression, et sans au moins une copie de ses sauvegardes qui ne dépende pas de l'instance.

Tester. Restaurer une sauvegarde une fois par trimestre, et vérifier que l'application démarre dessus : c'est le chapitre « Stockage bloc et fichiers », appliqué aux bases.

Définition : fenêtre de maintenance

Plage hebdomadaire pendant laquelle RDS peut appliquer les mises à jour du système et du moteur. Elle se choisit aux heures creuses.

Définition : déploiement bleu/vert (Blue/Green Deployment)

Fonction de RDS qui crée une copie synchronisée de la base, sur laquelle on applique le changement et on teste, avant de basculer en moins d'une minute.

Mise à jour	Comportement
Systeme d'exploitation	Annoncée à l'avance ; appliquée dans la fenêtre.
Version mineure du moteur	Automatique si l'option est activée ; compatible.
Version majeure du moteur	Jamais automatique : c'est une décision, à tester.

Monter de version majeure

1. Lire les changements incompatibles de la nouvelle version.
2. Restaurer une copie et y tester l'application.
3. Prendre un snapshot.
4. Basculer par déploiement bleu/vert, ou appliquer la mise à jour dans une fenêtre annoncée.

En Multi-AZ, la mise à jour est appliquée au secours, puis une bascule a lieu : la coupure se limite à une ou deux minutes.

Fin de support. Une version majeure arrivée en fin de vie doit être quittée : au-delà, AWS facture un support étendu, puis force la mise à jour.

Métrique CloudWatch	Ce qu'elle révèle
CPUUtilization	Requêtes coûteuses, classe trop petite.
FreeableMemory	Mémoire disponible : trop basse, la base lit sur disque.
FreeStorageSpace	Espace libre : à zéro, la base s'arrête.
DatabaseConnections	Connexions ouvertes : proche du maximum, les nouvelles sont refusées.
ReadIOPS, WriteIOPS, latences	Stockage sollicité au-delà de sa performance.
ReplicaLag	Retard d'un réplica.
CPUCreditBalance	Crédits restants d'une classe à crédits.

Trois alarmes minimales

- Espace libre sous 20 %.
- Processeur au-dessus de 80 % pendant 15 minutes.
- Connexions au-dessus de 80 % du maximum.

Définition : CloudWatch Database Insights

Outil d'analyse de la charge d'une base, successeur de Performance Insights : il montre quelles requêtes consomment le plus, et ce qui les fait attendre.

Journaux. Les journaux du moteur (erreurs, requêtes lentes) peuvent être publiés dans CloudWatch Logs. Le journal des requêtes lentes est le premier endroit où chercher une cause de lenteur.

Erreur	Conséquence	Correction
Rétention à zéro, ou laissée à un jour	Aucun retour possible au-delà de la veille.	7 jours au moins ; 35 pour une base critique.
Suppression sans snapshot final	Base perdue.	Protection contre la suppression ; snapshot final.
Croire qu'une restauration écrase l'instance	Surprise : une seconde instance, et l'application toujours sur l'ancienne.	Prévoir la redirection.
Restauration avec les valeurs par défaut	Mauvais security group, pas de Multi-AZ.	Préciser réseau et options à la restauration.
Restauration jamais essayée	RTO inconnu, procédure découverte en crise.	Exercice périodique.
Sauvegardes dans le seul compte de production	Détruites avec lui en cas de compromission.	Copie par AWS Backup dans un autre compte.
Snapshots manuels jamais supprimés	Coût de stockage croissant.	Nommage daté, revue.
Heure de restauration donnée en heure locale	Restauration deux heures trop tôt ou trop tard.	Toujours en UTC.

RDS, disponibilité et sauvegardes, partie 3

Aperçu d'Amazon Aurora

Le moteur relationnel conçu par AWS pour le cloud : compatible MySQL et PostgreSQL, avec un stockage d'une autre nature.

- Ce qu'est Aurora
- Le stockage partagé
- Ce qui change par rapport à RDS
- Aurora Serverless
- Les déclinaisons
- RDS ou Aurora

Définition : Amazon Aurora

Moteur de base de données relationnelle conçu par AWS, compatible avec MySQL ou avec PostgreSQL, dans lequel le calcul et le stockage sont séparés : les instances partagent un volume répliqué sur trois zones.

Caractéristique	Détail
Compatibilité	Mêmes pilotes, mêmes outils, même SQL que MySQL ou PostgreSQL.
Stockage	Six copies des données, réparties sur trois zones ; grandit seul, jusqu'à plus de 100 Tio.
Réplicas	Jusqu'à 15, qui lisent le même volume : retard de quelques millisecondes.
Bascule	Un réplica est promu, en général en moins de 30 secondes.
Sauvegardes	Continues, sans fenêtre ni effet sur la performance.
Gestion	Par la console et l'API de RDS.

Définition : cluster Aurora

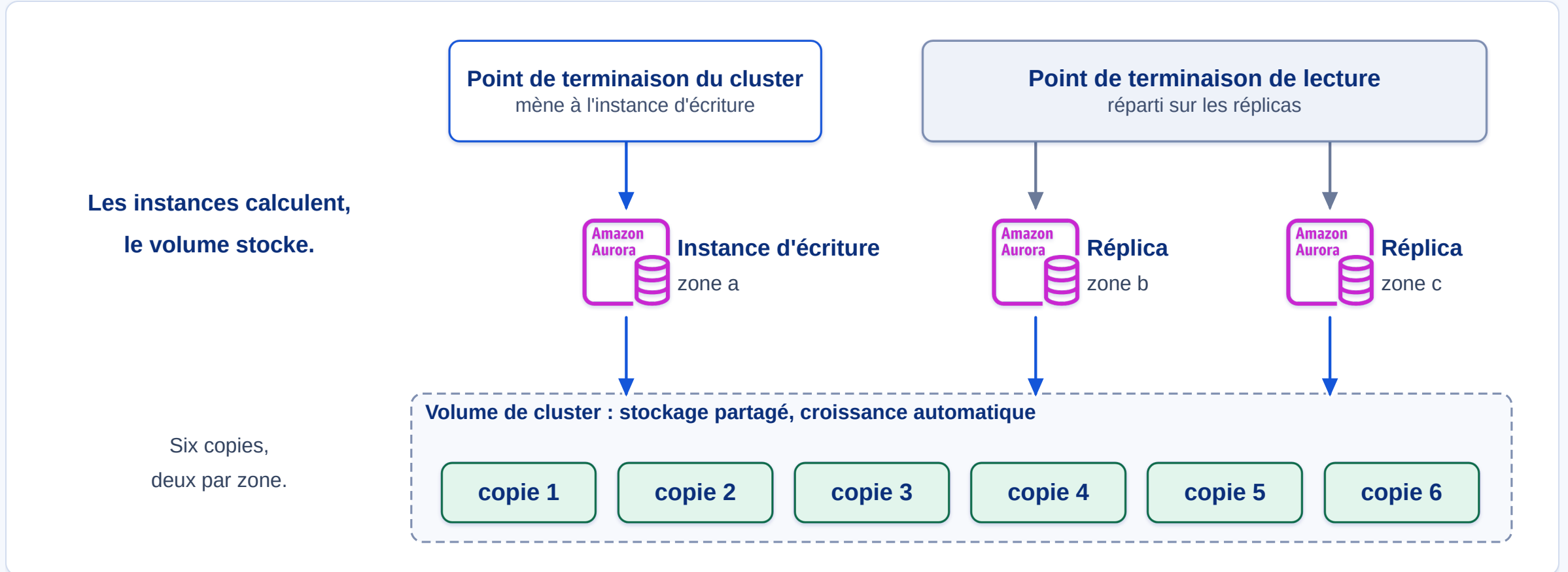
Ensemble formé d'un volume de stockage partagé et des instances qui s'y rattachent : une instance d'écriture, et des réplicas de lecture.

Vocabulaire

- On crée un **cluster**, puis des **instances** dans ce cluster.
- Le **point de terminaison du cluster** mène toujours à l'instance d'écriture.
- Le **point de terminaison de lecture** répartit les connexions entre les réplicas.

Un réplica Aurora fait les deux métiers : il sert les lectures et il est le secours. La distinction entre Multi-AZ et réplica de lecture disparaît.

Schéma : l'architecture d'un cluster Aurora



Lecture. Les instances ne possèdent pas de disque : toutes lisent le même volume, et l'instance d'écriture y écrit. Ajouter un réplica ne demande donc aucune copie de données : il est prêt en quelques minutes, quelle que soit la taille de la base.

Tolérance aux pannes du stockage. Avec six copies sur trois zones, le volume continue d'accepter les écritures après la perte d'une zone entière, et reste lisible après la perte d'une zone et d'une copie supplémentaire. Les copies endommagées se réparent seules.

Aurora

Ce qui change par rapport à RDS

	RDS (PostgreSQL, MySQL)	Aurora
Stockage	Volume EBS par instance, à dimensionner.	Volume partagé, croissance automatique, facturé à l'usage.
Copies des données	Deux en Multi-AZ.	Six, sur trois zones, toujours.
Réplicas	Réplication du moteur, asynchrone.	Lecture du même volume, retard minime.
Secours	Instance dédiée, inutilisée.	Un réplica, qui sert aussi à lire.
Bascule	Une à deux minutes.	En général moins de 30 secondes.
Capacité à la demande	Non.	Aurora Serverless.
Fonctions propres	Celles du moteur.	Clonage instantané, retour arrière (MySQL), base mondiale.
Prix	Le plus bas pour une petite base.	Instances un peu plus chères ; stockage et entrées-sorties facturés à part, ou forfaitisés.

Deux modes de facturation du stockage. « Standard » : stockage et entrées-sorties à l'usage, avantageux pour une base peu sollicitée. « I/O-Optimized » : entrées-sorties incluses, avantageux quand elles pèsent plus d'un quart de la facture.

Définition : Aurora Serverless v2

Mode d'Aurora dans lequel la capacité de calcul d'une instance s'ajuste en continu à la charge, entre un minimum et un maximum, et se facture à la seconde.

Définition : ACU (Aurora Capacity Unit)

Unité de capacité d'Aurora Serverless : environ 2 Gio de mémoire, avec le processeur et le réseau correspondants.

À savoir

- La capacité varie par pas d'une demi-unité, sans coupure.
- Avec un minimum à zéro, la base se met en pause quand elle ne reçoit plus de connexions, et reprend en quelques secondes.
- Idéal pour les charges irrégulières, les environnements de test, les applications nouvelles.

Déclinaison	Ce qu'elle apporte
Aurora provisionné	Des instances de classe fixe, comme RDS.
Aurora Serverless v2	Une capacité qui suit la charge.
Aurora Global Database	Un cluster principal et des copies en lecture dans d'autres régions, avec moins d'une seconde de retard.
Aurora DSQL	Base SQL distribuée, sans serveur, compatible PostgreSQL, active dans plusieurs régions à la fois.

Mélanger. Un même cluster peut réunir une instance d'écriture provisionnée et des réplicas Serverless, qui absorbent les pointes de lecture.

Serverless ne veut pas dire gratuit au repos si le minimum n'est pas à zéro : une demi-unité tourne en permanence. Et le stockage se paie toujours.

Situation	Choix
Petite base, charge modeste et stable	RDS : le moins cher.
Moteur Oracle, SQL Server, Db2, MariaDB	RDS : Aurora ne les propose pas.
Besoin d'une version ou d'une extension précise	RDS, souvent plus à jour sur ce point.
Forte charge de lecture, nombreux réplicas	Aurora.
Exigence de bascule très rapide	Aurora.
Charge irrégulière ou imprévisible	Aurora Serverless.
Base qui grossit vite, taille inconnue	Aurora : stockage automatique.
Présence dans plusieurs régions	Aurora Global Database.

Ce qui ne change pas

- Le réseau : groupe de sous-réseaux isolés, security group, pas d'accès public.
- Les identifiants : mot de passe géré par Secrets Manager.
- Le chiffrement, les snapshots, la restauration à un instant.
- La console, la CLI `aws rds`, AWS Backup.

Tout ce qui a été vu dans les chapitres « RDS, créer et se connecter » et « RDS, disponibilité et sauvegardes » s'applique.

Passer de l'un à l'autre est simple : un snapshot RDS MySQL ou PostgreSQL se restaure en cluster Aurora. On peut commencer par RDS et migrer quand le besoin apparaît.

Ce qu'il faut retenir

- Multi-AZ : une copie synchrone dans une autre zone, bascule automatique, même point de terminaison. Pour la disponibilité.
- Réplica de lecture : copie asynchrone, lisible, promotion manuelle. Pour la performance.
- Aucune réplication ne protège d'une erreur de manipulation.
- Sauvegardes automatiques : chaque jour, plus les journaux, sur 1 à 35 jours.
- Restauration à un instant donné : à la seconde, vers une nouvelle instance.
- Snapshot manuel : conservé jusqu'à suppression ; snapshot final avant de supprimer.
- Aurora : stockage partagé sur trois zones, réplicas qui servent aussi de secours, mode Serverless.

1. L'instance de secours d'un déploiement Multi-AZ est-elle lisible ?

2. Le point de terminaison change-t-il après une bascule ?

3. La réplication vers un réplica de lecture est-elle synchrone ?

4. Quelle est la rétention maximale des sauvegardes automatiques ?

5. Une restauration écrase-t-elle l'instance d'origine ?

6. Qu'est-ce qui protège d'un DELETE malencontreux ?

7. Combien de copies des données dans Aurora, et sur combien de zones ?

Cliquer sur une question pour afficher la réponse.

DynamoDB, partie 1

Amazon DynamoDB

Une base NoSQL sans serveur : des temps de réponse de quelques millisecondes, à n'importe quelle échelle, pourvu que l'on accède aux données par leur clé.

- NoSQL et clé-valeur
- Le service
- Table, item, attribut
- Clé de partition, clé de tri
- Modéliser d'après les accès
- Premières commandes

Définition : base NoSQL

Base de données qui ne suit pas le modèle relationnel : pas de schéma fixe ni de jointures, mais un modèle de données simple, conçu pour monter en charge horizontalement.

Définition : base clé-valeur

Base dans laquelle chaque donnée est rangée sous une clé, et retrouvée par cette clé. L'accès est extrêmement rapide, mais c'est le seul accès efficace.

Définition : Amazon DynamoDB

Base NoSQL clé-valeur et document, entièrement managée et sans serveur : aucune instance à choisir, une capacité et un stockage qui s'ajustent, une réplication d'office sur trois zones.

Propriété	Détail
Sans serveur	Ni instance, ni version, ni correctif, ni fenêtre de maintenance.
Performance	Quelques millisecondes, que la table contienne mille items ou mille milliards.
Échelle	Stockage et débit sans plafond pratique.
Disponibilité	Données répliquées sur trois zones, sans rien régler.
Accès	API HTTPS, contrôlée par IAM : pas de mot de passe, pas de connexion à maintenir.
Facturation	Aux lectures, aux écritures et au volume stocké.

Pas dans un VPC. Une table se joint par son API régionale, comme S3. Depuis un sous-réseau sans sortie, on passe par un endpoint passerelle, gratuit.

DynamoDB	Équivalent relationnel	Différence
Table	Table	Aucun schéma, sauf la clé.
Item	Ligne	Jusqu'à 400 Ko ; chaque item a ses propres attributs.
Attribut	Colonne	Peut contenir une liste ou un document imbriqué.
Clé primaire	Clé primaire	Seul moyen d'accès efficace.
Index secondaire	Index	Une autre clé pour les mêmes items.
Aucun	Jointure	N'existe pas : les données liées se rangent ensemble.

Définition : item

Enregistrement d'une table DynamoDB : un ensemble d'attributs, identifié de façon unique par sa clé primaire.

Type d'attribut	Exemples
Scalaire	Chaîne (S), nombre (N), binaire (B), booléen, nul.
Document	Liste (L), dictionnaire (M) : structures imbriquées, comme du JSON.
Ensemble	Ensemble de chaînes ou de nombres, sans doublon.

Sans schéma. Ajouter un attribut ne demande aucune migration : on l'écrit dans les nouveaux items. C'est au code de traiter les items qui ne l'ont pas.

client_id clé de partition	commande clé de tri	autres attributs libres, différents selon les items
C-4812	2026-09-12#A17	total = 59,90 statut = livrée
C-4812	2026-10-03#B02	total = 120,00 statut = préparée cadeau = oui
C-7730	2026-10-01#A99	total = 12,50 statut = livrée code_promo = RENTREE

Un item = une ligne. Seule la clé est imposée ; chaque item porte les attributs dont il a besoin.

Définition : clé de partition

Attribut obligatoire de la clé primaire. Sa valeur, passée par une fonction de hachage, décide de la partition où l'item est rangé.

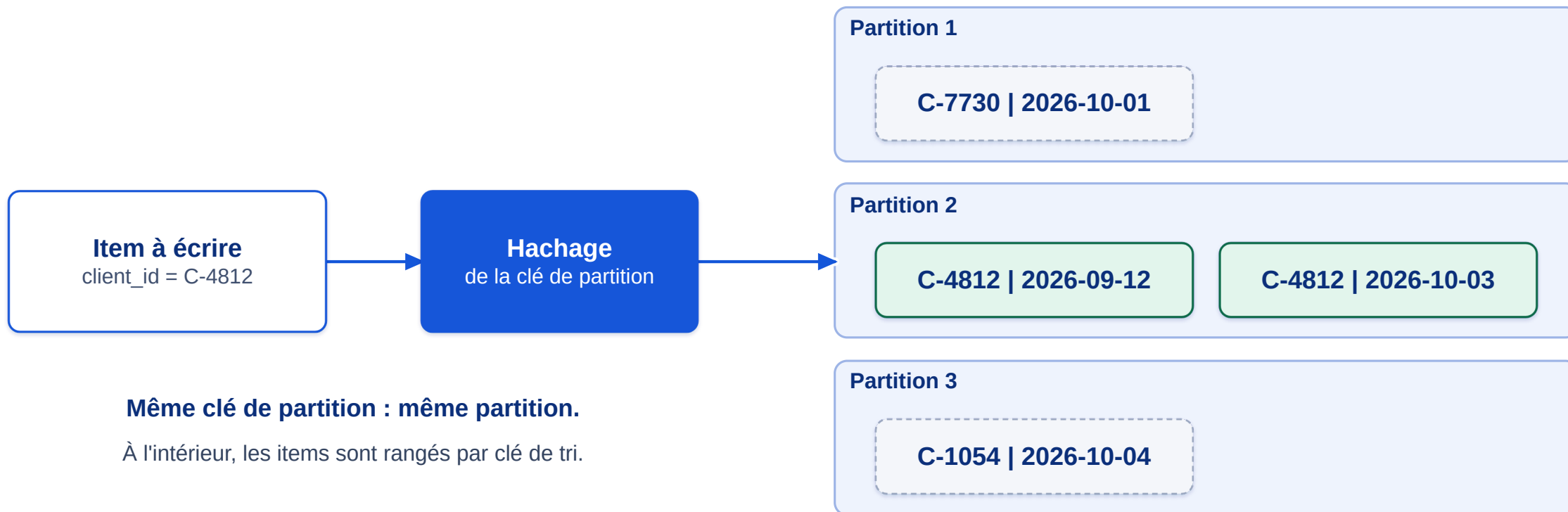
Définition : clé de tri

Second attribut, facultatif, de la clé primaire. Les items de même clé de partition sont rangés dans l'ordre de leur clé de tri, ce qui permet de les lire par plage.

	Clé simple	Clé composée
Composition	Clé de partition seule.	Clé de partition et clé de tri.
Unicité	Une valeur, un item.	Le couple est unique ; plusieurs items partagent la même clé de partition.
Lecture possible	Un item, par sa clé.	Un item ; ou tous les items d'une clé de partition ; ou une plage de clés de tri.
Exemple	Table <code>sessions</code> : <code>session_id</code> .	Table <code>commandes</code> : <code>client_id</code> et <code>commande</code> .
Usage	Accès direct : profil, session, configuration.	Relations « un à plusieurs » : les commandes d'un client, les messages d'une conversation.

La clé de tri porte souvent une date ou une hiérarchie. `2026-10-03#B02` : toutes les commandes d'octobre s'obtiennent en demandant les clés de tri qui commencent par `2026-10`.

La clé se choisit à la création et ne se change plus. Une table mal conçue ne se corrige pas : on en crée une autre et l'on y recopie les données. D'où l'importance de la diapositive suivante.



Même clé de partition : même partition.

À l'intérieur, les items sont rangés par clé de tri.

Définition : partition

Unité de stockage interne d'une table, répartie sur plusieurs serveurs. DynamoDB en ajoute automatiquement à mesure que la table grossit ou reçoit plus de trafic.

Pourquoi c'est rapide à toute échelle. Connaissant la clé de partition, DynamoDB sait directement sur quelle partition aller : il n'explore jamais la table. C'est aussi pourquoi une requête sans clé de partition l'oblige à tout lire.

Choisir une bonne clé de partition

Clé de partition	Verdict
<code>client_id</code> , <code>session_id</code> , <code>commande_id</code>	Bonne : beaucoup de valeurs, trafic bien réparti.
<code>statut</code> (trois valeurs)	Mauvaise : toute la table tient dans trois partitions.
<code>pays</code>	Mauvaise : un pays concentre l'essentiel du trafic.
<code>date du jour</code>	Mauvaise : toutes les écritures du jour au même endroit.

Définition : partition chaude (hot partition)

Partition qui reçoit une part disproportionnée du trafic, parce que trop de requêtes visent la même clé de partition. Elle atteint sa limite alors que le reste de la table est au repos.

Deux critères

- **Cardinalité élevée** : un très grand nombre de valeurs distinctes.
- **Accès uniforme** : aucune valeur ne concentre le trafic.

Et un troisième, évident mais décisif : c'est la valeur que l'application connaît au moment de lire.

Limites par clé de partition. Environ 3 000 lectures et 1 000 écritures par seconde pour une même valeur de clé. Au-delà, les requêtes sont rejetées, quelle que soit la capacité de la table.

	Relationnel	DynamoDB
Point de départ	Les données et leurs relations.	Les questions que posera l'application.
Démarche	Normaliser, puis écrire les requêtes.	Lister les accès, puis dessiner les clés.
Question imprévue	Une nouvelle requête SQL.	Un nouvel index, ou une refonte.

Le renversement. En relationnel, on modélise puis on interroge. Avec DynamoDB, on liste d'abord les accès, et c'est eux qui dictent les clés. Commencer par créer une table « pour voir » mène presque toujours à un Scan.

Accès de UtopiosShop	Réponse par les clés
Le panier d'un client	Clé de partition <code>client_id</code> .
Les commandes d'un client, les plus récentes d'abord	<code>client_id</code> , clé de tri commençant par la date, lecture en ordre inverse.
Une commande précise	<code>client_id</code> et clé de tri complète.
Les commandes à préparer	Index secondaire sur <code>statut</code> .
Le chiffre d'affaires par mois et par rayon	Pas pour DynamoDB : base relationnelle, ou export vers l'analytique.

Ranger ensemble ce qui se lit ensemble. Pas de jointure : si l'écran d'une commande affiche aussi le nom du client, ce nom est recopié dans l'item de la commande.

Définition : lecture à cohérence éventuelle

Lecture qui peut, pendant une fraction de seconde après une écriture, renvoyer l'ancienne valeur. C'est le mode par défaut, et le moins cher.

Définition : lecture fortement cohérente

Lecture qui renvoie toujours la dernière valeur écrite. Elle coûte le double et se demande explicitement.

Situation	Lecture
Afficher un catalogue, un profil	Cohérence éventuelle.
Relire un solde juste après l'avoir modifié	Fortement cohérente.

Écritures sûres

- **Écriture conditionnelle** : « écrire seulement si l'item n'existe pas », « seulement si le stock est positif ». Elle évite que deux requêtes simultanées s'écrasent.
- **Compteur atomique** : incrémenter un nombre sans le lire d'abord.
- **Transaction** : jusqu'à 100 opérations sur plusieurs items ou tables, toutes appliquées ou aucune. Coût doublé.

DynamoDB sait faire des transactions, mais sur des items désignés par leur clé. Ce n'est pas l'équivalent d'une transaction SQL qui parcourt et modifie des milliers de lignes liées.

aws

DynamoDB > Tables > Create table

Table name

user7-commandes

Partition key

client_id (String)

Sort key (optional)

commande (String)

Table settings

☒ Customize settings

Table class

☒ DynamoDB Standard

Capacity mode

☒ On-demand

Encryption at rest

☒ Owned by Amazon DynamoDB

Deletion protection

☒ Turn on

Create table

Réglage	À savoir
Clés	Nom et type : définitifs.
Capacity mode	À la demande pour commencer ; modifiable ensuite.
Encryption	Toujours chiffrée ; clé d'AWS par défaut, ou clé KMS à vous.
Deletion protection	À activer sur toute table de production.

Rien d'autre à déclarer. Les autres attributs n'existent pas au niveau de la table : ils apparaissent avec les items.

Explorer les données. Menu **Explore items** : parcourir, filtrer, modifier un item. Les requêtes s'y composent en choisissant « Query » ou « Scan » : la distinction est déjà là.

\$ Prérequis : AWS CLI connectée, région eu-west-3 ; remplacer user7

```
# 1. Créer la table : clé de partition client_id, clé de tri commande, capacité à la demande
aws dynamodb create-table --table-name user7-commandes \
  --attribute-definitions AttributeName=client_id,AttributeType=S AttributeName=commande,AttributeType=S \
  --key-schema AttributeName=client_id,KeyType=HASH AttributeName=commande,KeyType=RANGE \
  --billing-mode PAY_PER_REQUEST --deletion-protection-enabled
aws dynamodb wait table-exists --table-name user7-commandes

# 2. Écrire un item : chaque valeur est précédée de son type (S chaîne, N nombre, BOOL booléen)
aws dynamodb put-item --table-name user7-commandes --item '{
  "client_id": {"S": "C-4812"}, "commande": {"S": "2026-10-03#B02"},
  "total": {"N": "120.00"}, "statut": {"S": "preparee"}, "cadeau": {"BOOL": true} }'

# 3. Lire cet item par sa clé complète
aws dynamodb get-item --table-name user7-commandes \
  --key '{"client_id": {"S": "C-4812"}, "commande": {"S": "2026-10-03#B02"}}'

# 4. Modifier un attribut, seulement si l'item est encore « preparee » (écriture conditionnelle)
aws dynamodb update-item --table-name user7-commandes \
  --key '{"client_id": {"S": "C-4812"}, "commande": {"S": "2026-10-03#B02"}}' \
  --update-expression "SET statut = :n" --condition-expression "statut = :a" \
  --expression-attribute-values '{":n": {"S": "expediee"}, ":a": {"S": "preparee"}}'
```

HASH et RANGE sont les noms que l'API donne à la clé de partition et à la clé de tri. Dans une application, les SDK masquent la notation typée : on manipule des objets ordinaires.

DynamoDB, partie 2

Capacité : à la demande ou provisionnée

Payer chaque requête, ou réserver un débit : le choix dépend de ce que l'on sait de la charge.

- Les unités de lecture et d'écriture
- Le mode à la demande
- Le mode provisionné
- Les rejets
- Ce que cela coûte
- Choisir

Définition : unité d'écriture

Une écriture d'un item de 1 Ko au plus. Un item plus gros consomme une unité par kilo-octet entamé.

Définition : unité de lecture

Une lecture fortement cohérente d'un item de 4 Ko au plus, ou deux lectures à cohérence éventuelle. Un item plus gros consomme une unité par tranche de 4 Ko.

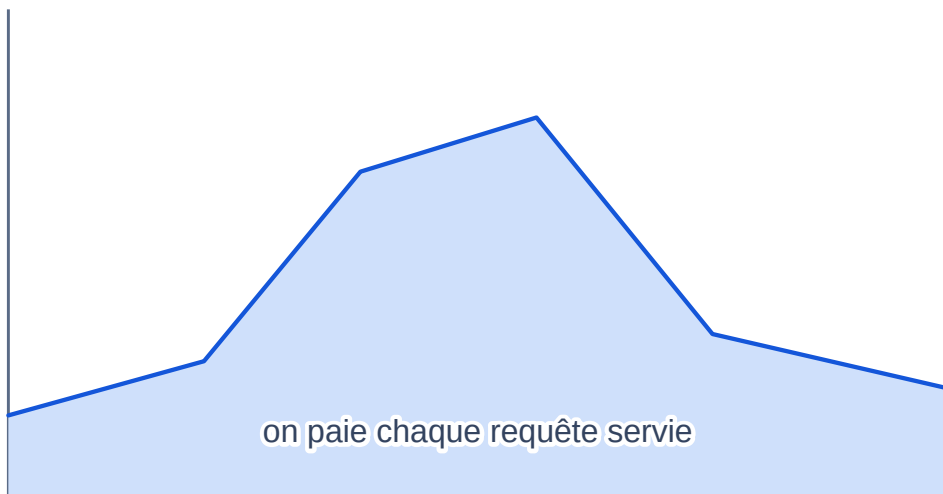
En mode provisionné, on parle d'unités **par seconde** : WCU et RCU. En mode à la demande, d'unités de requête : WRU et RRU.

Opération	Consommation
Écrire un item de 800 octets	1 unité d'écriture
Écrire un item de 3,5 Ko	4 unités d'écriture
Lire un item de 3 Ko, cohérence éventuelle	0,5 unité de lecture
Lire un item de 3 Ko, fortement cohérente	1 unité de lecture
Lire un item de 10 Ko, fortement cohérente	3 unités de lecture
Écriture dans une transaction	Le double

Des items petits. Le coût et la vitesse dépendent directement de la taille des items. Les gros contenus (images, documents) vont dans S3 ; l'item n'en garde que la clé.

Schéma : deux modes de capacité

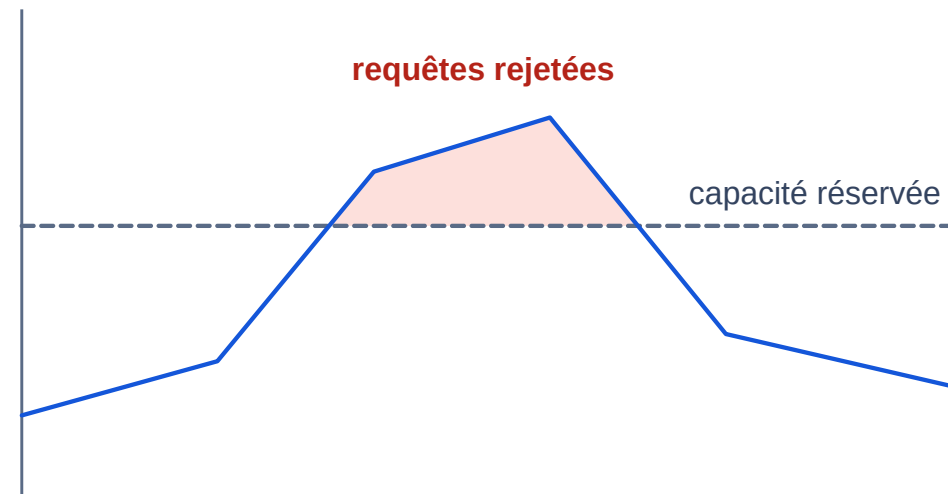
À la demande



La capacité suit la charge, sans réglage.

L'ajustement automatique déplace la ligne réservée, avec quelques minutes de retard.

Provisionnée



On paie la capacité réservée, utilisée ou non.

Définition : mode à la demande (on-demand)

Mode dans lequel la table absorbe le trafic qui se présente, sans réglage, et facture chaque lecture et chaque écriture.

Définition : mode provisionné

Mode dans lequel on réserve un nombre de lectures et d'écritures par seconde, facturé à l'heure qu'il soit utilisé ou non. Un ajustement automatique peut le faire suivre la charge.

Comparer les deux modes

	À la demande	Provisionné
Réglage	Aucun.	Débit de lecture et d'écriture, ou plages d'ajustement automatique.
Facturation	Par requête.	Par unité réservée et par heure.
Charge nulle	Coût nul, hors stockage.	La réservation reste due.
Pic soudain	Absorbé, jusqu'au double du pic précédent instantanément.	Rejets au-delà de la réservation, le temps que l'ajustement réagisse.
Coût à charge stable et soutenue	Plus élevé.	Nettement plus bas ; reservable sur un ou trois ans.
Convient à	Applications nouvelles, charge irrégulière ou inconnue, environnements de test.	Charge connue, régulière, prévisible.

Commencer à la demande. C'est le mode par défaut, et le bon tant que l'on ne connaît pas la charge. On passe en provisionné quand les mesures montrent un trafic régulier dont le coût le justifie.

Changer de mode est possible sur une table existante, sans coupure, avec une limite sur la fréquence des changements. Le choix n'est donc pas définitif.

Définition : limitation (throttling)

Rejet d'une requête par DynamoDB parce que le débit autorisé est dépassé. L'erreur porte le nom

`ProvisionedThroughputExceededException` ou

`ThrottlingException`.

Cause	Remède
Capacité provisionnée trop basse	L'augmenter ; ajustement automatique ; mode à la demande.
Partition chaude	Revoir la clé de partition : aucune capacité n'y remédie.
Index secondaire sous-dimensionné	Lui donner sa propre capacité suffisante.
Scan massif	Le remplacer par une Query, ou le ralentir.

Ce que fait le SDK

- Il réessaie de lui-même les requêtes rejetées, en espaçant les tentatives de plus en plus.
- Un rejet ponctuel passe donc inaperçu, au prix d'un peu de latence.
- Des rejets continus finissent par remonter en erreur dans l'application.

Définition : attente exponentielle (exponential backoff)

Stratégie de nouvelle tentative dans laquelle le délai double à chaque échec, avec une part d'aléa, pour ne pas aggraver la surcharge.

Surveiller. Les métriques `ThrottledRequests`, `ConsumedReadCapacityUnits` et `ConsumedWriteCapacityUnits` de CloudWatch ; l'outil « Contributor Insights » désigne les clés les plus sollicitées.

Capacité

Ce que coûte DynamoDB

Poste	Tarif indicatif
Écritures, à la demande	0,625 \$ par million
Lectures, à la demande	0,125 \$ par million
Écriture provisionnée	0,00065 \$ par unité et par heure
Lecture provisionnée	0,00013 \$ par unité et par heure
Stockage (classe Standard)	0,25 \$ par Go et par mois, après 25 Go gratuits
Sauvegarde continue, flux, tables mondiales	En supplément, s'ils sont activés

Exemple mensuel	Ordre de grandeur
5 millions d'écritures, 20 millions de lectures, 10 Go, à la demande	6 \$
50 écritures et 200 lectures par seconde en continu, provisionné	43 \$
Le même trafic, à la demande	148 \$

Lecture. Pour une petite application ou un trafic irrégulier, la demande coûte presque rien. Pour un trafic soutenu et constant, le provisionné revient trois à quatre fois moins cher.

Définition : classe de table
Standard, ou Standard-IA : stockage moins cher et requêtes plus chères, pour les tables volumineuses et peu consultées.

DynamoDB, partie 3

Requêtes, index et TTL

Une seule règle : désigner les données par leur clé. Tout le reste en découle.

- Les opérations de lecture
- Query contre Scan
- Le piège du filtre
- Les index secondaires
- Le TTL
- Sauvegardes et fonctions annexes

Opération	Ce qu'elle fait	Clé requise
GetItem	Lit un item.	Clé primaire complète.
PutItem	Crée ou remplace un item.	Clé primaire complète.
UpdateItem	Modifie certains attributs d'un item.	Clé primaire complète.
DeleteItem	Supprime un item.	Clé primaire complète.
Query	Lit les items d'une même clé de partition, tous ou une plage de clés de tri.	Clé de partition.
Scan	Parcourt toute la table.	Aucune.
BatchGetItem, BatchWriteItem	Regroupent jusqu'à 100 lectures ou 25 écritures en un appel.	Clé de chaque item.
TransactWriteItems, TransactGetItems	Plusieurs opérations, en tout ou rien.	Clé de chaque item.

PartiQL. DynamoDB accepte aussi une syntaxe proche de SQL (`SELECT ... WHERE client_id = 'C-4812'`). Elle ne change rien au moteur : sans la clé de partition dans la condition, la requête est un Scan.

Toutes les opérations sauf une exigent de connaître la clé. Scan est l'exception, et c'est précisément pour cela qu'il faut s'en méfier.

Schéma : Query contre Scan

Query : client_id = C-4812

partition 1

partition 2 : C-4812

partition 3

partition 4

Lit une seule partition, et seulement les items demandés.

Coût et durée proportionnels au résultat.

Scan : statut = préparée

partition 1

partition 2 : C-4812

partition 3

partition 4

Lit toute la table, puis jette ce qui ne correspond pas.

Coût et durée proportionnels à la taille de la table.

Définition : Query

Opération qui lit, dans une seule partition, les items dont la clé de partition est donnée, éventuellement restreints à une plage de clés de tri, dans l'ordre de cette clé.

Définition : Scan

Opération qui lit tous les items d'une table, page par page, et n'applique d'éventuels critères qu'après les avoir lus.

Ce que coûte la différence

	Query	Scan
Ce qui est lu	Les items d'une clé de partition.	Toute la table.
Durée	Millisecondes.	Proportionnelle à la taille : secondes, minutes, heures.
Coût	Celui des items renvoyés.	Celui de tous les items lus.
Ordre du résultat	Par clé de tri, croissant ou décroissant.	Aucun.
Effet sur les autres requêtes	Aucun.	Peut consommer toute la capacité.

Table de 10 Go, 20 millions d'items	Query	Scan
Unités de lecture consommées	1	1,3 million
Coût à la demande	négligeable	0,16 \$
Durée	5 ms	plusieurs minutes

Un Scan par affichage de page : 0,16 \$ et plusieurs minutes, à chaque visiteur. C'est l'erreur de conception la plus courante avec DynamoDB, et elle ne se voit pas tant que la table est petite.

Usages légitimes du Scan. Export, migration, traitement de toute la table, petite table de référence. Jamais sur le chemin d'une requête d'utilisateur.

\$ Prérequis : la table user7-commandes contient quelques items

1. Toutes les commandes d'un client

```
aws dynamodb query --table-name user7-commandes \  
  --key-condition-expression "client_id = :c" \  
  --expression-attribute-values '{"c": {"S": "C-4812"}}'
```

2. Seulement celles d'octobre 2026, les plus récentes d'abord, cinq au plus, deux attributs seulement

```
aws dynamodb query --table-name user7-commandes \  
  --key-condition-expression "client_id = :c AND begins_with(commande, :m)" \  
  --expression-attribute-values '{"c": {"S": "C-4812"}, "m": {"S": "2026-10"}}' \  
  --no-scan-index-forward --limit 5 --projection-expression "commande, total"
```

3. Combien d'unités de lecture cette requête a-t-elle consommées ?

```
aws dynamodb query --table-name user7-commandes \  
  --key-condition-expression "client_id = :c" \  
  --expression-attribute-values '{"c": {"S": "C-4812"}}' \  
  --return-consumed-capacity TOTAL --query 'ConsumedCapacity.CapacityUnits'
```

4. Le Scan équivalent, à éviter : il lit toute la table pour trouver les mêmes items

```
aws dynamodb scan --table-name user7-commandes \  
  --filter-expression "client_id = :c" --expression-attribute-values '{"c": {"S": "C-4812"}}' \  
  --return-consumed-capacity TOTAL --query '[Count,ScannedCount,ConsumedCapacity.CapacityUnits]'
```

Conditions sur la clé de tri. Égalité, comparaison, intervalle (`BETWEEN`), début (`begins_with`). Sur la clé de partition, l'égalité seule.

Pagination. Une réponse est limitée à 1 Mo. S'il reste des items, elle contient `LastEvaluatedKey` , à renvoyer pour obtenir la page suivante. La CLI et les SDK savent enchaîner.

Le piège de l'expression de filtre

Définition : expression de filtre

Critère appliqué par DynamoDB aux items déjà lus, avant de les renvoyer. Il réduit la réponse, pas la lecture : les items écartés sont tout de même facturés.

	Condition de clé	Filtre
Porte sur	La clé primaire.	N'importe quel attribut.
S'applique	Avant la lecture.	Après la lecture.
Réduit le coût	Oui.	Non.

L'illusion

Un Scan avec le filtre `statut = "preparee"` renvoie 12 items : la requête semble précise. Elle a pourtant lu les 20 millions d'items de la table.

Deux nombres le montrent dans la réponse : `Count` , les items renvoyés, et `ScannedCount` , les items lus. Quand le second dépasse de loin le premier, la requête est mal conçue.

La bonne réponse. Si l'on interroge régulièrement sur un attribut qui n'est pas la clé, il faut un index secondaire dont cet attribut est la clé.

Le filtre garde un usage : affiner une Query déjà bien ciblée, quand les items écartés sont peu nombreux.

Schéma : l'index secondaire global

Table commandes

clé de partition : client_id, clé de tri : date

C-4812 | 2026-10-03 | préparée

C-7730 | 2026-10-01 | livrée

C-1054 | 2026-10-04 | préparée

« Les commandes d'un client » : Query sur la table.

copie

automatique

Index secondaire global « par-statut »

clé de partition : statut, clé de tri : date

préparée | 2026-10-03 | C-4812

préparée | 2026-10-04 | C-1054

livrée | 2026-10-01 | C-7730

« Les commandes à préparer » : Query sur l'index.

Définition : index secondaire global (GSI)

Copie de tout ou partie d'une table, tenue à jour automatiquement, organisée selon une autre clé de partition et une autre clé de tri. Elle s'interroge comme une table, par Query.

Lecture. La table répond aux questions qui partent du client. L'index répond à celles qui partent du statut. Ce sont les mêmes items, rangés deux fois : une question nouvelle demande un rangement nouveau.

	Index secondaire global (GSI)	Index secondaire local (LSI)
Clé de partition	Différente de celle de la table.	La même que la table.
Clé de tri	Libre.	Différente de celle de la table.
Création	À tout moment.	Uniquement à la création de la table.
Nombre	Jusqu'à 20 par table.	Jusqu'à 5.
Cohérence des lectures	Éventuelle seulement : l'index suit la table avec un léger retard.	Forte possible.
Capacité	La sienne, distincte de celle de la table.	Celle de la table.
Usage	Interroger selon un tout autre critère.	Trier autrement les items d'une même clé de partition.

Définition : projection

Liste des attributs recopiés dans un index : les clés seules, certains attributs, ou tous. Moins on en recopie, moins l'index coûte.

Un index n'est pas gratuit. Chaque écriture dans la table est répétée dans chaque index qu'elle concerne : cinq index, c'est jusqu'à six écritures facturées. On en crée un par besoin d'accès réel, pas « au cas où ».

\$ Prérequis : la table user7-commandes existe, en mode à la demande ; ses items portent les attributs statut et commande

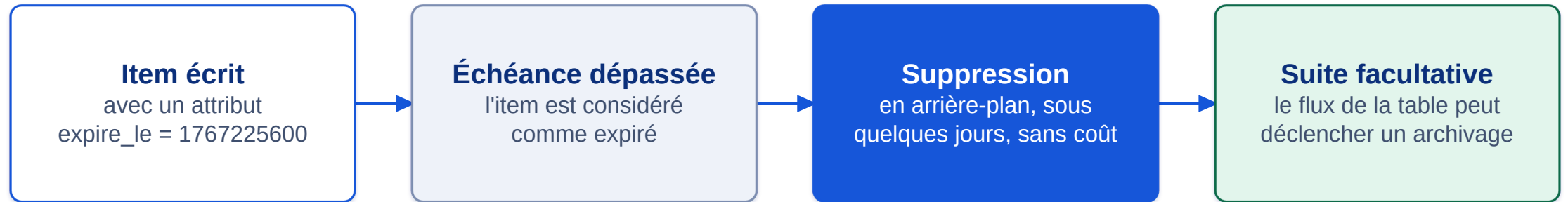
```
# 1. Ajouter un index secondaire global : clé de partition statut, clé de tri commande ; seul total est recopié en plus des clés
aws dynamodb update-table --table-name user7-commandes \
  --attribute-definitions AttributeName=statut,AttributeType=S AttributeName=commande,AttributeType=S \
  --global-secondary-index-updates '[{"Create": {"IndexName": "par-statut",
    "KeySchema": [{"AttributeName": "statut", "KeyType": "HASH"}, {"AttributeName": "commande", "KeyType": "RANGE"}],
    "Projection": {"ProjectionType": "INCLUDE", "NonKeyAttributes": ["total"]}}]]'
```

```
# 2. Attendre que l'index soit construit : CREATING, puis ACTIVE
aws dynamodb describe-table --table-name user7-commandes \
  --query 'Table.GlobalSecondaryIndexes[].[IndexName,IndexStatus,ItemCount]' --output table
```

```
# 3. Les commandes à préparer : une Query, sur l'index
aws dynamodb query --table-name user7-commandes --index-name par-statut \
  --key-condition-expression "statut = :s" --expression-attribute-values '{":s": {"S": "preparee"}}'
```

Index creux. Un item qui ne possède pas l'attribut servant de clé à l'index n'y figure pas. Astuce courante : ne poser l'attribut `a_preparer` que sur les commandes en attente ; l'index ne contient alors qu'elles, et reste minuscule.

Attention à la clé de l'index. `statut` n'a que quelques valeurs : sur une très grosse table, c'est une partition chaude. On la combine alors avec autre chose, par exemple le statut et le jour.



La valeur est une date au format Unix, en secondes.

Entre l'échéance et la suppression effective, l'item peut encore apparaître : filtrer à la lecture.

Définition : TTL (durée de vie) d'un item

Fonction qui supprime automatiquement les items dont un attribut, contenant une date, est dépassé. La suppression est gratuite et ne consomme aucune capacité d'écriture.

\$ Activer le TTL sur l'attribut expire_le

```
aws dynamodb update-time-to-live \  
  --table-name user7-sessions \  
  --time-to-live-specification \  
    "Enabled=true,AttributeName=expire_le"
```

```
# Échéance dans 24 heures, au format attendu  
echo $(( $(date +%s) + 86400 ))
```

Fonction	Ce qu'elle fait	Usage
Restauration à un instant donné	Sauvegarde continue ; retour à n'importe quelle seconde des 35 derniers jours, vers une nouvelle table.	À activer sur toute table de production.
Sauvegarde à la demande	Copie complète, conservée jusqu'à suppression ; intégrée à AWS Backup.	Archivage, avant une migration.
Protection contre la suppression	Empêche de supprimer la table.	Production.
DynamoDB Streams	Flux ordonné de toutes les modifications, conservé 24 heures.	Déclencher une fonction Lambda à chaque changement ; alimenter un autre système.
Tables mondiales	Même table dans plusieurs régions, en écriture dans chacune.	Utilisateurs sur plusieurs continents ; reprise après sinistre.
Export vers S3	Copie de la table dans S3, sans consommer sa capacité.	Analyse, à la place d'un Scan.
DAX	Cache en mémoire devant DynamoDB.	Lectures répétées exigeant moins d'une milliseconde.

Chiffrement et accès. Toute table est chiffrée au repos. L'accès se règle par IAM, jusqu'au niveau de l'item : une politique peut limiter un utilisateur aux items dont la clé de partition est son propre identifiant.

Erreur	Conséquence	Correction
Modéliser comme en relationnel	Une table par entité, puis des Scan pour les relier.	Partir des accès ; ranger ensemble ce qui se lit ensemble.
Scan sur le chemin d'une requête	Lenteur et coût croissants avec la table.	Query, sur la table ou sur un index.
Filtre pris pour une condition de clé	Requête « précise » qui lit tout.	Comparer <code>Count</code> et <code>ScannedCount</code> .
Clé de partition à peu de valeurs	Partition chaude, rejets.	Clé à forte cardinalité.
Gros documents dans les items	Coût élevé, limite de 400 Ko.	Contenu dans S3, clé dans l'item.
Index créés « au cas où »	Écritures multipliées.	Un index par accès réel.
Lecture sur un index juste après l'écriture	Item pas encore visible.	Cohérence éventuelle : le prévoir.
TTL attendu à la seconde	Items expirés encore renvoyés.	Filtrer sur la date à la lecture.
Rapports et statistiques dans DynamoDB	Scan permanents.	Export vers S3, ou base relationnelle.

Ce qu'il faut retenir

- DynamoDB est une base clé-valeur sans serveur : rapide à toute échelle, à condition d'accéder par la clé.
- La clé de partition décide de l'emplacement ; la clé de tri, de l'ordre à l'intérieur.
- On modélise d'après les accès, pas d'après les données.
- À la demande pour une charge inconnue ou irrégulière ; provisionné pour une charge stable.
- Query lit une partition ; Scan lit toute la table.
- Un filtre ne réduit pas ce qui est lu, donc pas ce qui est payé.
- Un index secondaire range les mêmes items selon une autre clé ; le TTL les fait expirer.

1. Quelle est la taille maximale d'un item ?

2. À quoi sert la clé de tri ?

3. Qu'est-ce qu'une partition chaude ?

4. Quel mode de capacité pour une application nouvelle ?

5. Pourquoi éviter Scan ?

6. Un filtre réduit-il le coût d'une requête ?

7. Comment interroger sur un attribut qui n'est pas la clé ?

Cliquer sur une question pour afficher la réponse.

Choisir sa base, protéger ses secrets, partie 1

Choisir sa base de données

Il n'y a pas de meilleure base : il y a celle qui correspond à la forme des données et aux questions qu'on leur pose.

- Les familles de bases
- Relationnel ou NoSQL : les critères
- RDS et DynamoDB face à face
- Utiliser plusieurs bases
- ElastiCache
- Le cache devant la base

Famille	Faite pour	Service AWS	Exemple
Relationnelle	Données liées, requêtes variées, transactions.	RDS, Aurora	Commandes, comptabilité, stock.
Clé-valeur	Accès par clé, très fort débit, grande échelle.	DynamoDB	Paniers, sessions, profils.
En mémoire	Réponses en moins d'une milliseconde.	ElastiCache, MemoryDB	Cache, classements, compteurs.
Document	Documents JSON aux structures variées.	DocumentDB, DynamoDB	Catalogues, contenus.
Graphe	Relations nombreuses à parcourir.	Neptune	Réseaux sociaux, détection de fraude.
Séries temporelles	Mesures horodatées, en très grand nombre.	Timestream	Capteurs, métriques.
Recherche	Recherche en texte intégral, analyse de journaux.	OpenSearch Service	Moteur de recherche d'un site.
Entrepôt de données	Analyses sur de très gros volumes.	Redshift, Athena	Tableaux de bord, statistiques.

Dans cette formation, on retient les trois premières : elles couvrent l'immense majorité des applications. Les autres répondent à des besoins précis, qu'il faut savoir reconnaître.

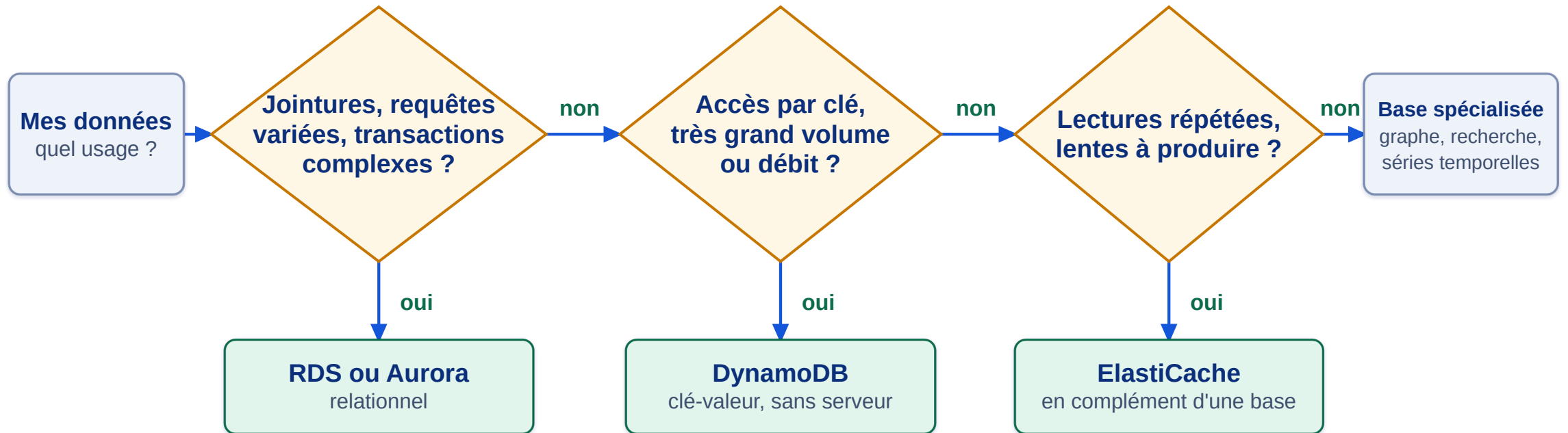
Tous sont managés. La question n'est donc plus « que sait-on exploiter ? », mais « quel modèle convient à ces données ? ».

Critère	Penche vers le relationnel	Penche vers DynamoDB
Forme des données	Entités liées entre elles, structure stable.	Items autonomes, structure variable.
Questions posées	Variées, changeantes, parfois imprévues.	Peu nombreuses, connues d'avance, toujours par clé.
Jointures, agrégations	Nécessaires.	Inutiles, ou faites ailleurs.
Transactions	Complexes, sur de nombreuses lignes.	Simple, sur quelques items.
Volume et débit	Jusqu'à plusieurs téraoctets et quelques milliers de requêtes par seconde.	Sans limite pratique.
Profil de charge	Régulier.	Très variable, pics brutaux, ou nul par moments.
Exploitation	Une instance à dimensionner et à surveiller.	Rien.
Compétences de l'équipe	SQL, connu de tous.	Modélisation par les accès, à apprendre.

Dans le doute : relationnel. Il pardonne les erreurs de conception et répond aux questions que l'on n'avait pas prévues. DynamoDB se choisit pour une raison précise : échelle, débit, absence totale d'exploitation, charge très irrégulière.

La question décisive. « Sais-je aujourd'hui, précisément, comment mes données seront lues ? » Si oui, DynamoDB est envisageable. Si non, une base relationnelle.

Schéma : choisir un type de base



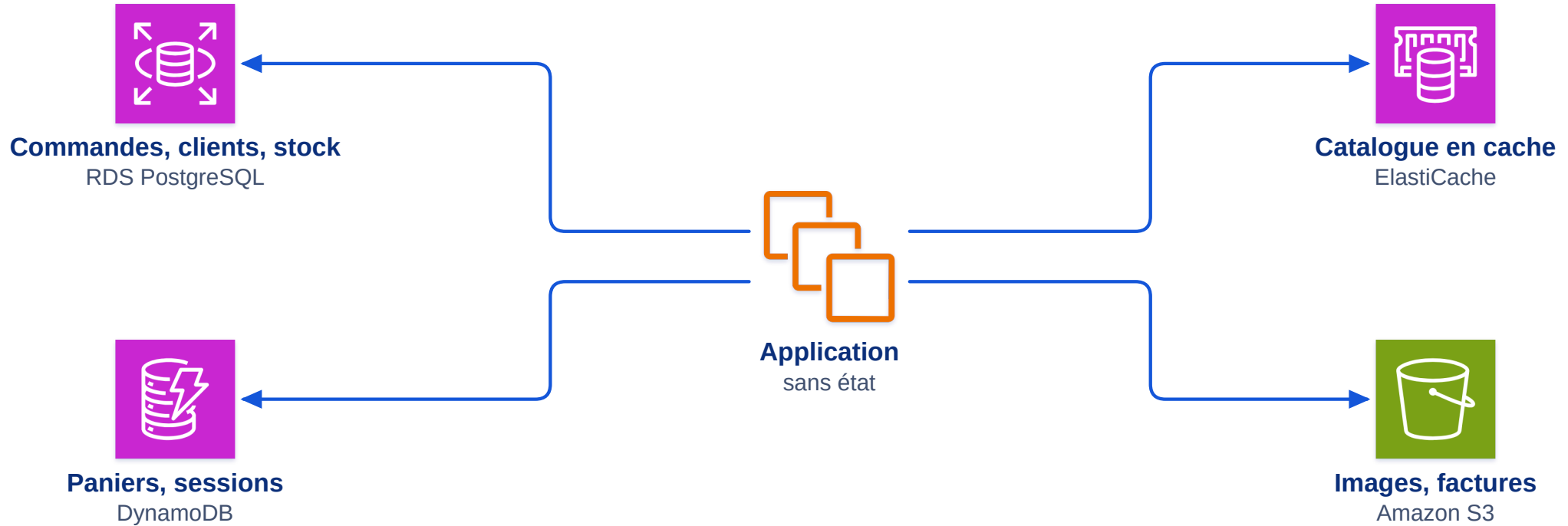
Lecture. L'arbre suit l'ordre des questions : d'abord la nature des requêtes, ensuite l'échelle, enfin la répétition des lectures. Le cache n'est pas une alternative aux deux autres : il vient en plus d'une base.

Idées reçues. « NoSQL est plus rapide » : seulement pour l'accès par clé. « Sans schéma, donc plus simple » : le schéma se déplace dans le code. « Relationnel ne monte pas en charge » : il suffit à la grande majorité des applications.

	Amazon RDS, Aurora	Amazon DynamoDB
Modèle	Tables liées, schéma défini.	Items identifiés par une clé, sans schéma.
Langage	SQL.	API (GetItem, Query...), PartiQL.
Requête imprévue	Possible ; un index l'accélère.	Scan, ou nouvel index.
Montée en charge	Classe plus grosse, réplicas de lecture.	Automatique, horizontale.
Disponibilité	Multi-AZ à activer.	Trois zones, d'office.
Réseau	Dans le VPC : sous-réseaux, security group.	API régionale, contrôlée par IAM.
Authentification	Compte et mot de passe de la base.	Rôle IAM.
Connexions	Persistantes, en nombre limité.	Aucune : requêtes HTTPS.
Coût à l'arrêt	L'instance tourne.	Nul à la demande, hors stockage.
Sauvegarde	Automatique, restauration à un instant.	À activer, restauration à un instant.

Lien avec le sans-serveur. Pas de connexion à maintenir, pas de VPC, authentification par rôle : DynamoDB s'accorde naturellement avec les fonctions Lambda, là où une base relationnelle demande RDS Proxy.

Schéma : plusieurs bases pour une application

**Définition : persistance polyglotte**

Fait d'utiliser, dans une même application, plusieurs types de bases de données, chacun pour les données auxquelles il convient le mieux.

Avec mesure. Chaque base ajoutée, c'est un modèle à maîtriser, des sauvegardes à vérifier, et une cohérence à assurer entre elles. On commence par une base relationnelle ; on en ajoute une seconde quand un besoin précis le justifie.

Définition : Amazon ElastiCache

Service managé de stockage de données en mémoire, compatible avec Valkey, Redis OSS et Memcached. Il sert de cache devant une base, ou de magasin de données très rapides.

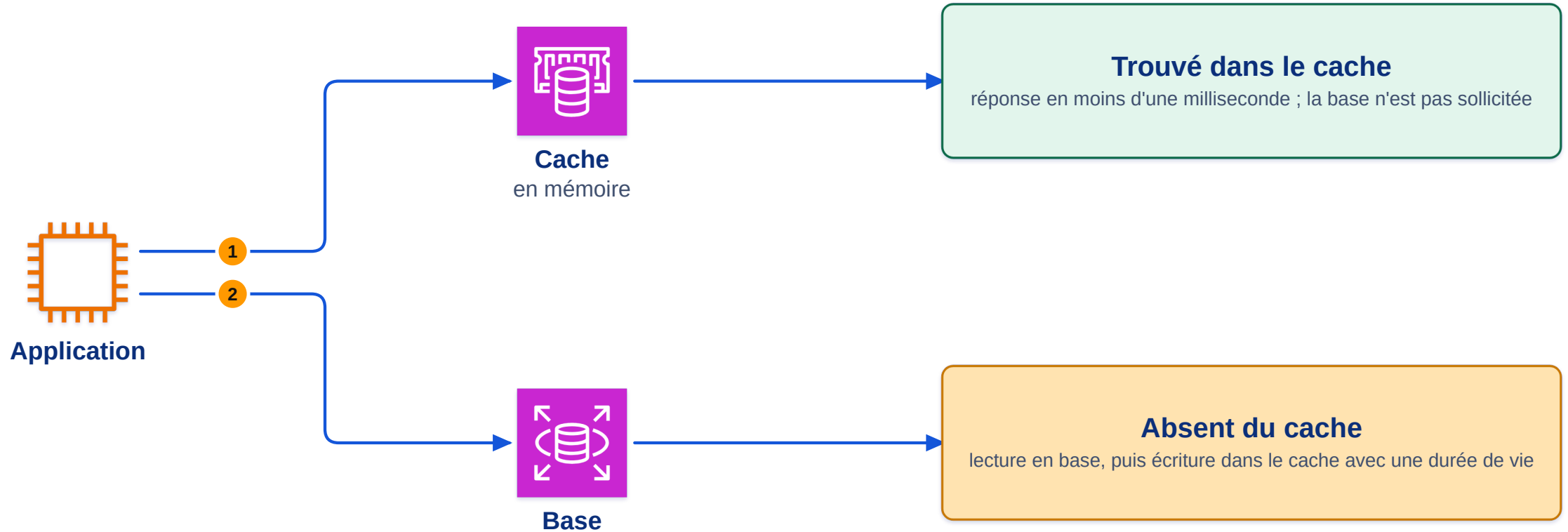
Moteur	À savoir
Valkey	Successeur libre de Redis, recommandé pour les nouveaux projets et le moins cher. Structures riches, réplication, persistance.
Redis OSS	Même famille ; pour les applications existantes.
Memcached	Cache simple, clé-valeur, sans réplication.

Deux formes. Serverless : aucune taille à choisir, facturé à l'usage.
Nœuds : un type de machine, des réplicas sur plusieurs zones. Dans les deux cas : dans le VPC, sous-réseaux privés, security group sur le port 6379.

Usage	Exemple
Cache de base de données	Résultat d'une requête coûteuse, gardé cinq minutes.
Sessions	L'état des utilisateurs, hors des instances.
Compteurs, limites	Nombre d'appels par client et par minute.
Classements	Meilleures ventes, scores.
Files, messages	Diffusion entre instances.

C'est le lieu des sessions évoqué au chapitre « Élasticité » : l'état que l'on sort des instances pour les rendre jetables.

Schéma : le cache devant la base



1. L'application cherche d'abord la donnée dans le cache.

2. Si elle n'y est pas, elle la lit dans la base.

3. Puis elle écrit la donnée dans le cache, avec une durée de vie : les demandes suivantes seront servies depuis la mémoire.

Définition : mise en cache à la demande (cache-aside)

Stratégie dans laquelle l'application consulte le cache, et ne lit la base qu'en cas d'absence, en alimentant alors le cache. C'est la plus courante.

Question	Réponse
Combien de temps garder ?	Une durée de vie sur chaque entrée : secondes pour un stock, heures pour un catalogue.
Et si la donnée change ?	Supprimer l'entrée à l'écriture en base, ou accepter le décalage jusqu'à expiration.
Et si le cache est vide ?	L'application fonctionne, plus lentement : elle lit la base.

Ce qui mérite d'être mis en cache

- Lu souvent, modifié rarement.
- Coûteux à produire : requête lourde, appel à un service lent.
- Tolérant à un léger décalage.

Le cache n'est pas la source de vérité. Il peut être vidé, redémarré, perdu : rien ne doit n'exister que dans le cache. Pour une donnée en mémoire qui doit survivre, il existe MemoryDB, ou la base elle-même.

Avant d'ajouter un cache. Vérifier les index et les requêtes : un cache devant une requête mal écrite masque le problème et en crée un second, la cohérence.

S'entraîner : quelle base pour quel besoin

1. Les commandes d'une boutique, avec leurs lignes, les clients et le stock.

2. Le panier des visiteurs, des millions par jour, lu par identifiant.

3. Les sessions d'utilisateurs d'un site derrière un Auto Scaling group.

4. La page des meilleures ventes, recalculée par une requête de dix secondes.

5. Le tableau de bord du chiffre d'affaires par mois et par région.

6. Les relevés de 100 000 capteurs, toutes les secondes.

7. La recherche « chaise bois scandinave » dans le catalogue.

8. Une application existante sur Oracle.

9. Un nouveau service dont le trafic est nul la nuit et imprévisible le jour.

10. Les photos des produits.

Cliquer sur une question pour afficher la réponse.

Erreur	Conséquence	Correction
Choisir DynamoDB « parce que c'est moderne »	Requêtes impossibles, Scan partout.	Partir des accès ; relationnel dans le doute.
Tout mettre dans une base relationnelle, images comprises	Base énorme, sauvegardes lentes.	Fichiers dans S3.
Rapports lourds sur la base de production	Application ralentie pour tous.	Réplica de lecture, ou entrepôt.
Cache ajouté avant d'avoir regardé les index	Problème masqué, données périmées en plus.	Optimiser d'abord la requête.
Entrées de cache sans durée de vie	Données périmées indéfiniment, mémoire saturée.	Une durée de vie sur chaque entrée.
Donnée unique gardée dans le cache	Perdue au premier redémarrage.	Le cache se reconstruit ; la vérité est dans la base.
Une base différente par microservice, par principe	Exploitation et cohérence ingérables.	Ajouter une base quand un besoin le justifie.
Base installée à la main pour un besoin que couvre un service managé	Charge d'exploitation inutile.	Vérifier le catalogue d'abord.

Choisir sa base, protéger ses secrets, partie 2

Protéger ses secrets et ses clés

Trois services, trois rôles : ranger les secrets, ranger la configuration, et garder les clés qui chiffrent le tout.

- Trois services, trois rôles
- Secrets Manager et la rotation
- Parameter Store
- Lequel choisir
- AWS KMS et le chiffrement d'enveloppe
- L'application et ses secrets

Schéma : trois services, trois rôles



Secrets Manager

mots de passe, clés d'API
rotation automatique

contient : des secrets



Parameter Store

paramètres de configuration
et secrets simples

contient : des valeurs



AWS KMS

les clés de chiffrement
qui protègent les deux autres

contient : des clés

Lecture. Secrets Manager et Parameter Store conservent des valeurs que les applications lisent. KMS ne conserve que des clés, et ne les montre jamais : il chiffre et déchiffre pour le compte des autres services, ces deux-là compris.

Le principe commun. Rien de sensible dans le code, les images, les variables en clair ni le user data. L'application reçoit un **rôle**, et ce rôle lui donne le droit de lire, au moment voulu, ce dont elle a besoin.

Notion	Détail
Secret	Un nom, une valeur chiffrée (texte ou JSON, jusqu'à 64 Ko), des étiquettes.
Version	Chaque changement de valeur crée une version.
Étiquettes de version	<code>AWSCURRENT</code> : la valeur en vigueur. <code>AWSPREVIOUS</code> : la précédente. <code>AWSPENDING</code> : celle en cours de rotation.
Politique de ressource	Attachée au secret : permet le partage avec un autre compte.
Suppression	Différée de 7 à 30 jours, annulable pendant ce délai.

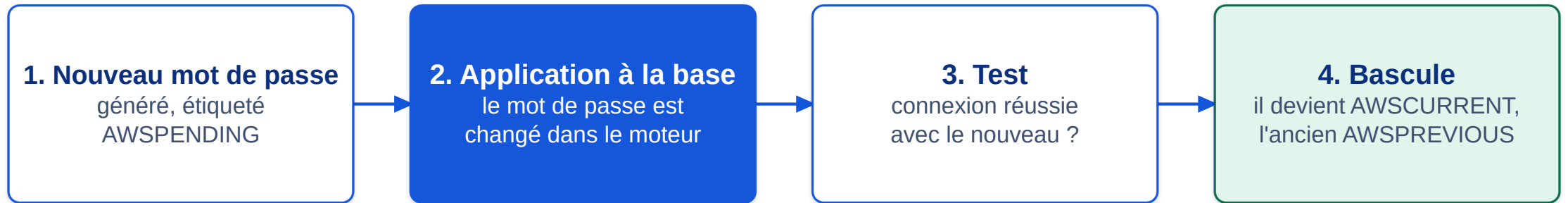
Définition : rotation d'un secret

Renouvellement automatique et programmé de la valeur d'un secret, à la fois dans Secrets Manager et dans le système qu'il protège, sans interruption de service.

Type de secret	Rotation
Mot de passe principal RDS ou Aurora géré	Entièrement prise en charge.
Autres comptes de base de données	Par une fonction Lambda fournie par AWS.
Clé d'API d'un service tiers	Par une fonction Lambda que vous écrivez.

Pourquoi faire tourner. Un secret qui change toutes les semaines limite à quelques jours la portée d'une fuite, et oblige à ne le lire qu'à un seul endroit.

Schéma : la rotation en quatre étapes



L'application qui relit le secret obtient toujours la version courante.

La rotation est exécutée par le service, ou par une fonction Lambda que vous fournissez.

Sans coupure. Pendant la rotation, l'ancien mot de passe reste valable jusqu'à ce que le nouveau ait été testé. Pour les applications qui ne tolèrent aucun échec de connexion, la rotation « à deux utilisateurs » alterne entre deux comptes de la base.

Condition. Les applications doivent relire le secret, et non garder indéfiniment une valeur lue au démarrage. Une application qui met le mot de passe en cache pour toujours tombe en panne à la première rotation.

Définition : **Parameter Store**

Fonction de AWS Systems Manager qui conserve des paramètres de configuration sous forme de paires nom-valeur, organisées en arborescence, dont certaines peuvent être chiffrées.

Type	Contenu
String	Une valeur en clair : URL, nom, option.
StringList	Une liste de valeurs séparées par des virgules.
SecureString	Une valeur chiffrée par une clé KMS.

txt Une arborescence de paramètres	
/shop/prod/db/host	String
/shop/prod/db/name	String
/shop/prod/api/timeout	String
/shop/prod/paiement/cle	SecureString
/shop/test/db/host	String

Niveau	Limites et coût
Standard	10 000 paramètres, 4 Ko chacun. Gratuit.
Avancé	8 Ko, davantage de paramètres, politiques d'expiration. 0,05 \$ par paramètre et par mois.

Chaque paramètre garde l'historique de ses valeurs. Il n'y a pas de rotation automatique.

Ce qu'apporte l'arborescence

- Lire en un appel tous les paramètres d'un chemin :
/shop/prod/ .
- Donner à un rôle l'accès à un seul sous-arbre : l'application de test ne lit pas la production.
- Garder la même structure d'un environnement à l'autre : seul le préfixe change.

	Secrets Manager	Parameter Store
Vocation	Les secrets.	La configuration, et les secrets simples.
Chiffrement	Toujours.	Au choix : type SecureString.
Rotation automatique	Oui.	Non.
Intégration avec RDS	Mot de passe principal géré.	Aucune.
Partage entre comptes	Par politique de ressource.	Possible pour les paramètres avancés.
Réplication entre régions	Oui.	Non.
Taille maximale	64 Ko.	4 Ko, ou 8 Ko.
Organisation	Noms libres.	Arborescence.
Coût	0,40 \$ par secret et par mois.	Gratuit au niveau standard.

Répartition habituelle. Dans Secrets Manager : ce qui doit tourner, mots de passe de base et clés d'API. Dans Parameter Store : tout le reste de la configuration, et les secrets qui ne changent pas souvent.

Passerelle. Parameter Store sait renvoyer un secret de Secrets Manager par un nom de paramètre particulier : une application peut tout lire par une seule API.

Définition : AWS KMS (Key Management Service)

Service qui crée et garde des clés de chiffrement dans des modules matériels sécurisés, et effectue pour vous les opérations de chiffrement, sans jamais laisser sortir les clés.

Type de clé	Qui la gère	Coût
Détenue par AWS	AWS, entièrement. Invisible dans votre compte.	Gratuite.
Gérée par AWS (<code>aws/rds</code> , <code>aws/s3</code> ...)	AWS, pour un service, dans votre compte. Politique non modifiable.	Gratuite ; requêtes facturées.
Gérée par le client	Vous : politique, rotation, désactivation, partage.	1 \$ par mois ; requêtes facturées.

Ce que KMS apporte

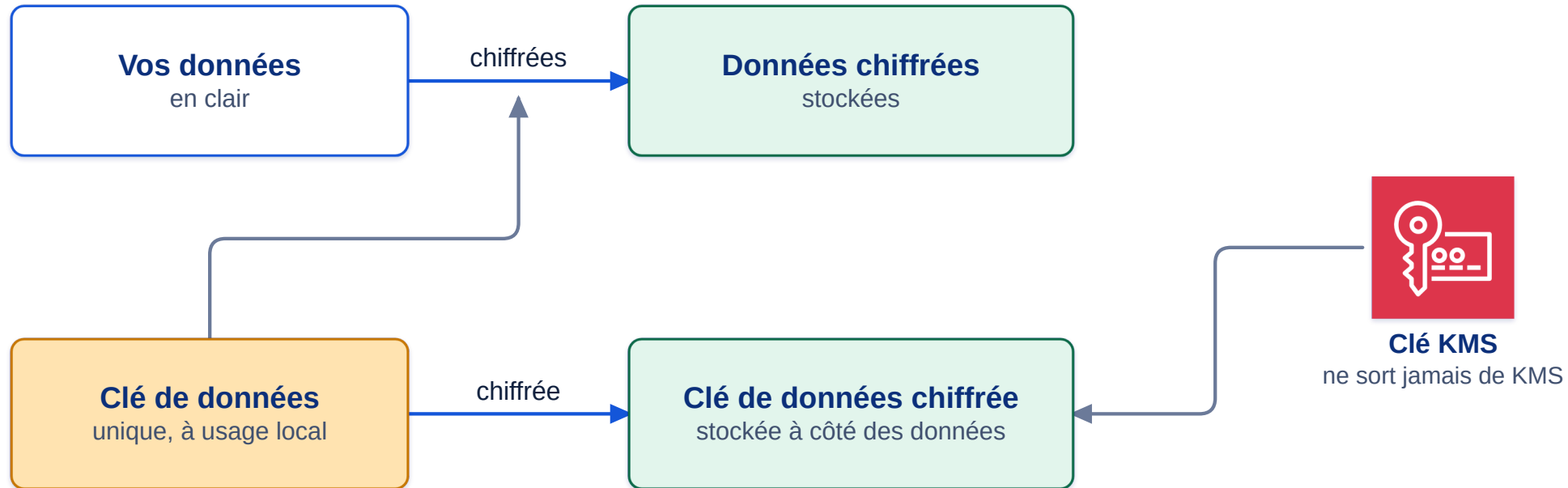
- **Un contrôle d'accès de plus** : lire une donnée chiffrée demande le droit sur la donnée et le droit d'utiliser la clé.
- **Une trace** : chaque usage d'une clé figure dans CloudTrail.
- **Un interrupteur** : désactiver une clé rend illisible, d'un coup, tout ce qu'elle protège.
- **Une rotation** annuelle automatique, sans rien rechiffrer.

Définition : politique de clé

Politique de ressource obligatoire d'une clé KMS. Elle seule peut autoriser l'usage de la clé ; IAM ne suffit pas si elle ne le permet pas.

Quand prendre une clé gérée par le client. Partage de données chiffrées avec un autre compte, exigence de maîtrise ou d'audit, séparation entre ceux qui gèrent les données et ceux qui gèrent les clés. Sinon, la clé gérée par AWS suffit.

Schéma : le chiffrement d'enveloppe



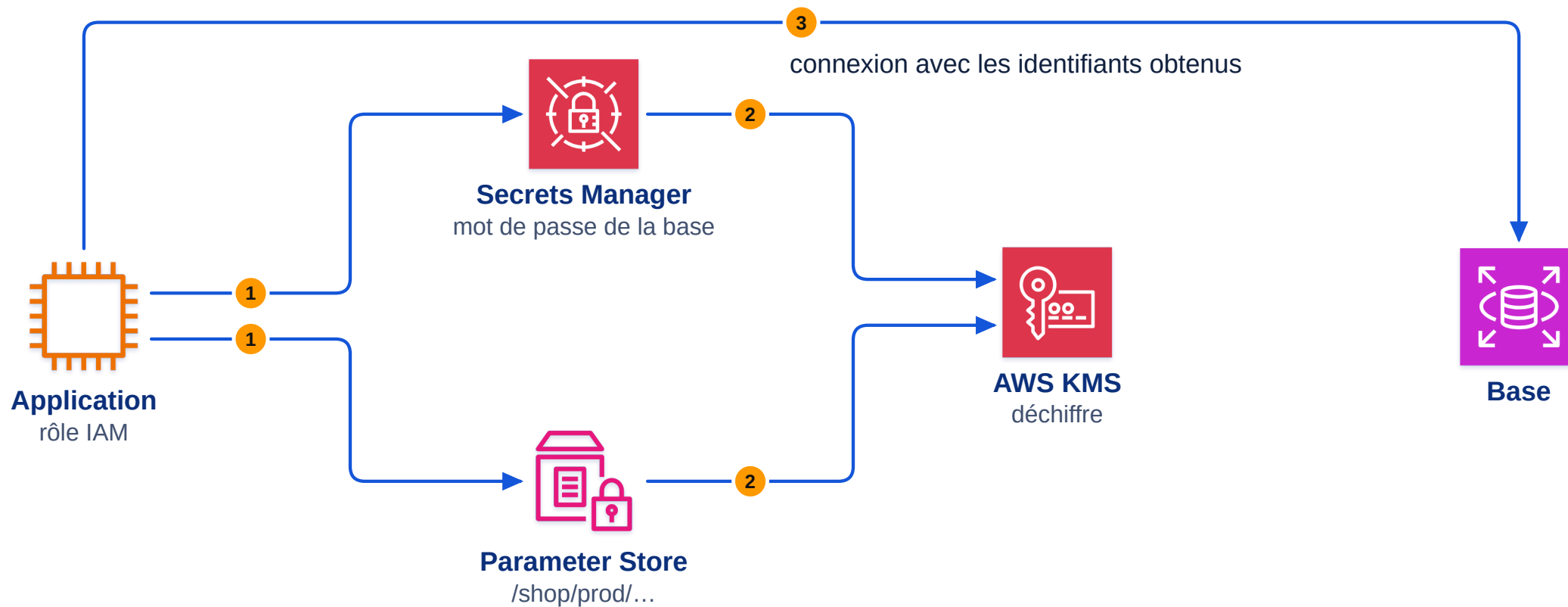
La clé de données chiffre les données ; la clé KMS chiffre la clé de données. Pour lire : le chemin inverse.

Définition : chiffrement d'enveloppe

Technique dans laquelle les données sont chiffrées par une clé de données, elle-même chiffrée par une clé KMS. On stocke ensemble les données chiffrées et la clé de données chiffrée.

Pourquoi ce détour. KMS ne chiffre directement que de petits volumes, et chaque appel a un coût. Avec une clé de données, les gigaoctets sont chiffrés sur place, vite, et un seul petit échange avec KMS suffit par objet. C'est ainsi que travaillent S3, EBS, RDS et DynamoDB.

Schéma : l'application et ses secrets



1. Au démarrage, l'application lit ses paramètres et ses secrets : son rôle IAM l'y autorise, pour ces noms-là seulement.

2. Les deux services font déchiffrer les valeurs par KMS ; le rôle doit donc aussi pouvoir utiliser la clé, si elle est gérée par le client.

3. L'application se connecte à la base avec les identifiants obtenus, qu'elle n'a jamais stockés.

\$ Prérequis : AWS CLI connectée ; remplacer user7

```
# --- Parameter Store ---
aws ssm put-parameter --name /user7/shop/prod/db/host --type String --value "$ENDPOINT"
aws ssm put-parameter --name /user7/shop/prod/paiement/cle --type SecureString --value "sk_test_exemple"
aws ssm get-parameter --name /user7/shop/prod/paiement/cle --with-decryption --query Parameter.Value --output text
aws ssm get-parameters-by-path --path /user7/shop/prod/ --recursive --with-decryption \
  --query 'Parameters[][Name,Type]' --output table

# --- Secrets Manager ---
aws secretsmanager create-secret --name user7/shop/prod/api-partenaire \
  --secret-string '{"client_id":"abc","client_secret":"exemple"}'
aws secretsmanager get-secret-value --secret-id user7/shop/prod/api-partenaire --query SecretString --output text
aws secretsmanager put-secret-value --secret-id user7/shop/prod/api-partenaire \
  --secret-string '{"client_id":"abc","client_secret":"nouvelle-valeur"}'
aws secretsmanager delete-secret --secret-id user7/shop/prod/api-partenaire --recovery-window-in-days 7

# --- KMS : une clé gérée par le client, un alias, la rotation annuelle ---
KEY_ID=$(aws kms create-key --description "Donnees de UtopiosShop" --query KeyMetadata.KeyId --output text)
aws kms create-alias --alias-name alias/user7-shop --target-key-id $KEY_ID
aws kms enable-key-rotation --key-id $KEY_ID
aws kms schedule-key-deletion --key-id $KEY_ID --pending-window-in-days 7
```

Attention à l'historique du terminal. Une valeur passée sur la ligne de commande y reste. Pour un vrai secret : le lire depuis un fichier (`--secret-string file://secret.json`), ou laisser le service le générer. Les valeurs ci-dessus sont des exemples.

Fournir les secrets aux applications

Environnement	Comment le secret arrive
Instance EC2	L'application appelle l'API avec le rôle de l'instance, au démarrage.
Tâche ECS	La définition de tâche référence le secret ; ECS l'injecte dans une variable du conteneur, grâce au rôle d'exécution.
Pod Kubernetes (EKS)	Un composant monte les secrets comme des fichiers dans le pod.
Fonction Lambda	La fonction appelle l'API, ou passe par l'extension de cache fournie par AWS.
Chaîne d'intégration	Le rôle de la chaîne lit le secret au moment du déploiement.

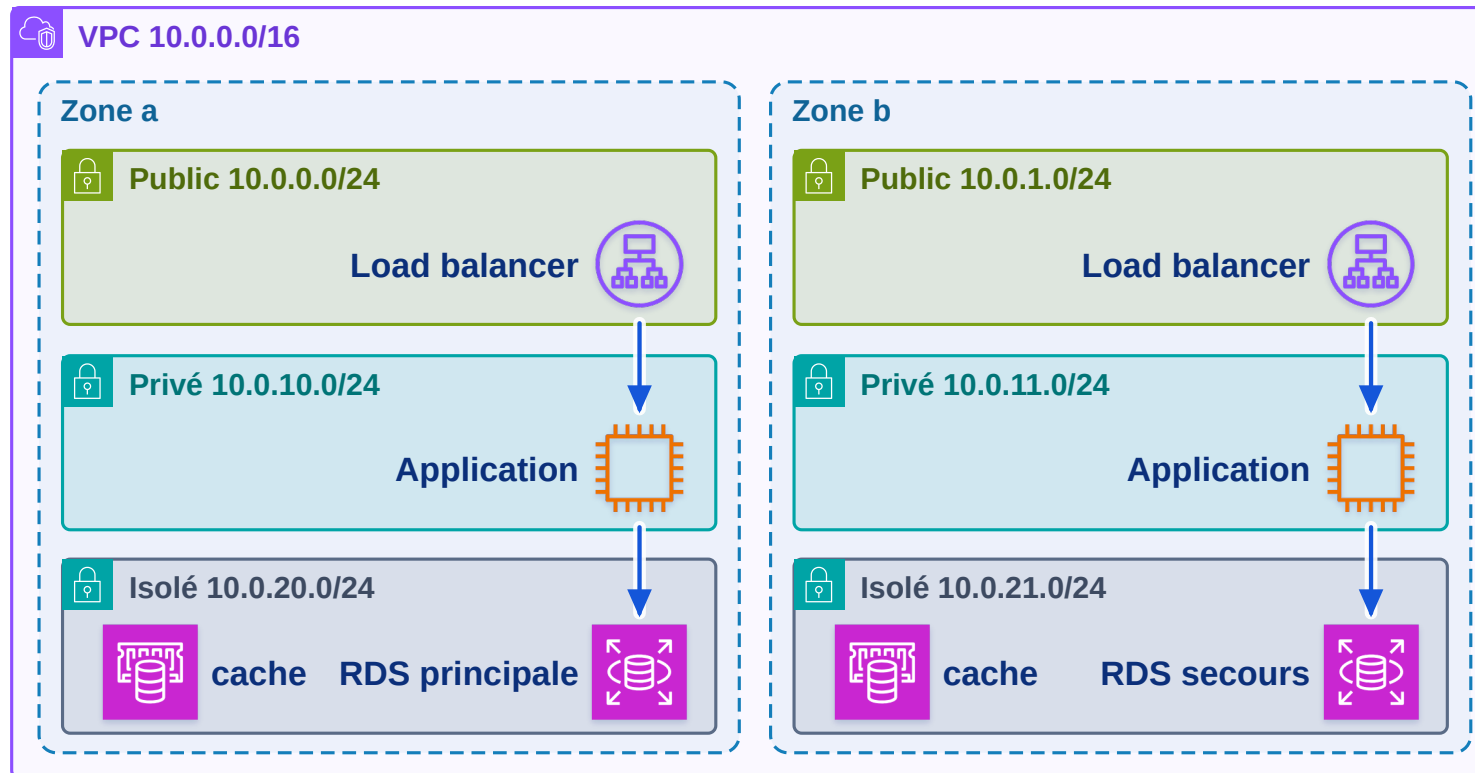
`{}` Politique du rôle : lire un secret et un sous-arbre de paramètres, rien d'autre

```
{ "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource":
        "arn:aws:secretsmanager:*:*:secret:shop/prod/*" },
    { "Effect": "Allow",
      "Action": ["ssm:GetParameter",
                 "ssm:GetParametersByPath"],
      "Resource":
        "arn:aws:ssm:*:*:parameter/shop/prod/*" }
  ] }
```

Moindre privilège. Le rôle de l'application de production ne lit que `shop/prod` . Une application compromise n'accède qu'à ses propres secrets.

Erreur	Conséquence	Correction
Secret dans le code ou le dépôt Git	Fuite durable : l'historique le conserve.	Secrets Manager ; faire tourner tout secret qui a fui.
Secret dans une variable d'environnement en clair	Visible dans la console et les descriptions.	Référence à un secret, injectée à l'exécution.
Droit <code>GetSecretValue</code> sur <code>*</code>	Une application lit les secrets de toutes les autres.	Limiter à un préfixe.
Secret lu une fois et gardé pour toujours	Panne à la rotation.	Relecture périodique, ou à l'échec.
Secret lu à chaque requête	Coût et latence.	Cache de quelques minutes.
Droit sur la clé KMS oublié	« Access Denied » alors que le secret est autorisé.	Ajouter <code>kms:Decrypt</code> sur la clé.
Clé KMS supprimée	Données définitivement illisibles.	Désactiver plutôt que supprimer ; délai d'attente maximal.
Mêmes secrets en test et en production	Une fuite en test ouvre la production.	Un jeu par environnement, sous des chemins distincts.
Secret affiché dans les journaux	Fuite par CloudWatch Logs.	Ne jamais journaliser une valeur lue.

Schéma : UtopiosShop, les données et leurs secrets



DynamoDB

paniers, sessions



Secrets Manager

mot de passe géré



AWS KMS

chiffrement au repos

Relationnel

Une base RDS PostgreSQL en Multi-AZ, dans les sous-réseaux isolés, chiffrée, sauvegardée 35 jours, que seule l'application peut joindre.

Clé-valeur et cache

Paniers et sessions dans DynamoDB, à la demande, avec un TTL ; les lectures coûteuses dans ElastiCache.

Secrets

Mot de passe de la base géré par Secrets Manager, configuration dans Parameter Store, tout chiffré par KMS, lu par le rôle de l'application.

Ce qu'il faut retenir

- Le choix d'une base part de la forme des données et des questions posées.
- Relationnel : données liées, requêtes variées, transactions. Le choix par défaut.
- DynamoDB : accès par clé, grande échelle, charge irrégulière, aucune exploitation.
- ElastiCache : un cache en mémoire, en complément d'une base, jamais à sa place.
- Secrets Manager range les secrets et les fait tourner.
- Parameter Store range la configuration, en arborescence.
- KMS garde les clés ; lire une donnée chiffrée demande aussi le droit sur la clé.

1. Quelle base choisir quand les accès ne sont pas encore connus ?

2. Le cache peut-il être la seule copie d'une donnée ?

3. Que faut-il sur chaque entrée de cache ?

4. Quel service fait tourner un mot de passe automatiquement ?

5. Où ranger l'URL d'un service et un délai d'attente ?

6. Qu'est-ce que le chiffrement d'enveloppe ?

7. Comment une application obtient-elle ses secrets ?

Cliquer sur une question pour afficher la réponse.

#	Pratique	Chapitre
1	Une base managée plutôt qu'une base installée sur une instance.	RDS, créer et se connecter
2	Base dans les sous-réseaux isolés, sans accès public, security group limité à l'application.	RDS, créer et se connecter
3	Chiffrement dès la création ; TLS imposé.	RDS, créer et se connecter
4	Mot de passe principal géré ; un compte applicatif aux droits limités.	RDS, créer et se connecter
5	Multi-AZ en production ; bascule essayée.	RDS, disponibilité et sauvegardes
6	Rétention des sauvegardes fixée explicitement ; protection contre la suppression ; restauration testée.	RDS, disponibilité et sauvegardes
7	DynamoDB : partir des accès, clé de partition à forte cardinalité, jamais de Scan en production.	DynamoDB
8	Capacité à la demande pour commencer ; sauvegarde continue activée.	DynamoDB
9	La base choisie pour ses données ; un cache avec durée de vie, jamais source de vérité.	Choisir sa base, protéger ses secrets
10	Aucun secret dans le code ; lecture par rôle, limitée à un préfixe.	Choisir sa base, protéger ses secrets
11	Alarmes sur l'espace libre, le processeur, les connexions et les rejets.	RDS, disponibilité et sauvegardes ; DynamoDB

Fil conducteur. Les données sont l'actif le plus précieux et le plus difficile à reconstituer : on les confie à des services qui les répliquent et les sauvegardent, on les isole du réseau, et l'on ne laisse traîner aucune clé qui y mène.

Synthèse

Glossaire (1/3) : RDS et Aurora

Terme	Définition
Base relationnelle	Tables liées, schéma défini, interrogation en SQL.
Instance de base de données	Un moteur, sur une machine d'une classe donnée, avec son stockage.
Point de terminaison	Nom DNS stable par lequel on joint la base.
Classe d'instance	Processeur et mémoire de la base.
Groupe de sous-réseaux	Sous-réseaux où RDS peut placer l'instance.
Groupe de paramètres	Réglages du moteur.
Multi-AZ	Copie synchrone dans une autre zone, bascule automatique.
Réplica de lecture	Copie asynchrone, en lecture seule.
Sauvegarde automatique	Sauvegarde quotidienne et journaux, sur 1 à 35 jours.
Restauration à un instant	Retour à une seconde précise, vers une nouvelle instance.
Snapshot final	Snapshot pris à la suppression d'une instance.
RDS Proxy	Mutualisation des connexions devant la base.
Aurora	Moteur d'AWS à stockage partagé sur trois zones.
Aurora Serverless	Capacité de calcul qui suit la charge.

Terme	Définition
NoSQL	Base hors du modèle relationnel, conçue pour l'échelle.
Table, item, attribut	Conteneur, enregistrement, champ.
Clé de partition	Attribut dont le hachage fixe l'emplacement de l'item.
Clé de tri	Attribut qui ordonne les items d'une même clé de partition.
Partition chaude	Partition qui concentre le trafic.
Unité de lecture, d'écriture	Mesure de la consommation : 4 Ko lus, 1 Ko écrit.
À la demande	Facturation par requête, sans réglage.
Provisionné	Débit réservé, facturé à l'heure.
Limitation (throttling)	Rejet d'une requête au-delà du débit autorisé.
Query	Lecture des items d'une clé de partition.
Scan	Parcours de toute la table.
Expression de filtre	Critère appliqué après la lecture.
Index secondaire global	Mêmes items, rangés selon une autre clé.
TTL	Expiration automatique des items.
Streams	Flux des modifications d'une table.

Terme	Définition
ElastiCache	Stockage en mémoire managé : Valkey, Redis OSS, Memcached.
Cache-aside	Lire le cache ; à défaut la base, puis alimenter le cache.
Durée de vie d'une entrée	Temps avant expiration d'une donnée en cache.
Persistance polyglotte	Plusieurs types de bases dans une même application.
Secret	Information dont dépend un accès.
Secrets Manager	Conservation et rotation des secrets.
Rotation	Renouvellement automatique d'un secret.
Parameter Store	Paramètres de configuration en arborescence.
SecureString	Paramètre chiffré par KMS.
AWS KMS	Garde et usage des clés de chiffrement.
Clé gérée par le client	Clé KMS dont vous maîtrisez la politique.
Politique de clé	Politique de ressource obligatoire d'une clé KMS.
Chiffrement d'enveloppe	Clé de données chiffrée par une clé KMS.
Authentification IAM	Connexion à la base par jeton temporaire.

Sujet	Documentation officielle
Amazon RDS	Amazon RDS User Guide
Amazon Aurora	Amazon Aurora User Guide
DynamoDB	Amazon DynamoDB Developer Guide
ElastiCache	Amazon ElastiCache User Guide
Secrets Manager	AWS Secrets Manager User Guide
AWS KMS	AWS KMS Developer Guide
Tarifs	aws.amazon.com/rds/pricing

Icônes. Les icônes des schémas proviennent du jeu officiel « AWS Architecture Icons », publié par AWS pour la réalisation de schémas d'architecture.

Ce qui est acquis

- Créer une base relationnelle managée, s'y connecter sans l'exposer.
- La rendre disponible, la sauvegarder, la restaurer.
- Modéliser et interroger une table DynamoDB.
- Choisir une base, et sortir tous les secrets du code.

Suite du parcours : les conteneurs

- Le registre d'images.
- Faire tourner des conteneurs sans gérer de serveurs.
- Kubernetes managé.

L'application de UtopiosShop, jusqu'ici sur des instances, y sera empaquetée en conteneurs.

Avant de quitter. Supprimer les instances RDS et leurs réplicas (snapshot final inutile pour un essai), les snapshots manuels, les tables, les caches, les secrets et les clés KMS créés pour l'exercice.